

Predicción de series de tiempo aplicado al trading de criptomonedas usando la arquitectura de Transformers

Ing. Martín Leonardo Centurión

Carrera de Especialización en Inteligencia Artificial

Director: Dr. Camilo Argoty

Jurados:

Jurado 1 (pertenencia)

Jurado 2 (pertenencia)

Jurado 3 (pertenencia)

Ciudad de Buenos Aires, Marzo de 2026

Resumen

Este trabajo se enfoca en el estudio y la aplicación de la arquitectura Transformer para la predicción de series de tiempo financieras, específicamente en el contexto de un sistema de trading de criptomonedas. Se desarrolla un modelo Transformer capaz de procesar indicadores técnicos para optimizar decisiones de compra y venta en el par BTC/USDT. Se aplican conceptos de aprendizaje profundo, redes neuronales y análisis de series de tiempo para la optimización de estrategias algorítmicas.

Índice general

Resumen	I
1. Introducción general	1
1.1. Motivación	1
1.2. Descripción de la problemática	2
1.3. Estado del arte	3
1.4. El desafío de los datos financieros	3
1.5. Objetivos y alcance	4
2. Introducción específica	7
2.1. Trading y análisis técnico	7
2.2. Aprendizaje supervisado	7
2.3. Optimización de hiperparámetros	8
2.4. Transformers	8
2.5. Metodologías de evaluación y gestión de riesgo	9
2.5.1. Técnicas de selección de características	9
2.5.2. Técnicas de etiquetado	10
2.5.3. Backtesting	10
2.5.4. Validación cruzada	10
2.5.5. Dimensionamiento de la apuesta	11
2.5.6. Ratio de Sharpe y sus variantes	11
3. Diseño e implementación	13
3.1. Conjunto de datos y preprocesamiento	13
3.1.1. Conjunto de datos	13
3.1.2. Estrategias de preprocesamiento	13
3.1.3. Ingeniería de características y análisis de importancia	15
3.2. Estrategias de referencia	15
3.3. Arquitectura transformer propuesta	16
3.4. Estrategia de etiquetado	17
3.5. Framework de backtesting y evaluación	18
3.5.1. Optimización de hiperparámetros: búsqueda bayesiana con Optuna	18
3.6. Modelos explorados	19
3.7. Diseño del bot	20
Fase de inicio y orquestación	21
Fase de ejecución continua	23
Fase de monitoreo remoto	25
Fase de apagado	26
3.8. Diseño del sistema	27
3.9. Pipeline de modelos	28
3.10. Estructura de pruebas realizadas con Chronos	30

4. Ensayos y resultados	33
4.1. Datos utilizados	33
4.2. Pruebas con algoritmos de trading	34
4.3. Pruebas con algoritmos estadísticos clásicos	36
4.4. Pruebas con algoritmos de machine learning	37
4.5. Pruebas con Chronos	39
5. Conclusiones	43
5.1. Conclusiones generales	43
5.2. Próximos pasos	43
Bibliografía	45

Índice de figuras

1.1. Evolución del precio del par BTC/USDT en el período analizado. ¹	2
1.2. Diagrama conceptual de la arquitectura del sistema de trading. . .	5
2.1. Representación de alto nivel de Chronos. Fase de tokenización (izquierda), entrenamiento (centro), e infrencia (derecha).	9
3.1. Diagrama del pipeline de preprocesamiento de datos.	15
3.2. Aplicación del método de etiquetado de triple barrera sobre un gráfico de precios.	17
3.3. Diagrama del proceso de optimización de hiperparámetros con Optuna.	19
3.4. Diagrama del proceso de inicialización del bot.	21
3.5. Diagrama del proceso de inicialización del bot.	21
3.6. Diagrama del proceso de inicialización del bot.	22
3.7. Diagrama del bucle de principal del bot de trading.	24
3.8. Diagrama del proceso de monitoreo del bot de trading.	26
3.9. Diagrama del proceso de apagado del bot de trading.	27
3.10. Flujo de datos en distintas etapas de procesamiento del pipeline. .	29
3.11. Estructura jerárquica de herencia de clase para AbstractMLPipeline.	30
3.12. Estructura jerárquica de herencia de clase para los experimentos realizados con Chronos.	30
4.1. Comparativa de barras de volumen y precio de cierre con frecuencia de 1m.	34
4.2. Superposición de precio de cierre cada 1m, barras de volumen a 50 000 unidades de BTC, y volumen de BTC.	34

Índice de tablas

4.1. Resumen Buy and Hold	33
4.2. Resultados de Optimización	35
4.3. Distribución de Sharpe	35
4.4. Consistencia Sharpe	35
4.5. Sensibilidad de Parámetros	36
4.6. Resultados de Optimización	36
4.7. Sensibilidad de Parámetros	36
4.8. Distribución de F1	37
4.9. Experimentos de machine learning	38
4.10. Distribución F1 por algoritmo	38
4.11. Familias de pipeline Chronos	39
4.12. Experimentos de embedding Chronos	39
4.13. Distribución F1 embedding Chronos	40
4.14. Resultados de pronóstico directo Chronos	40
4.15. Resultados de meta-etiquetado Chronos	41
4.16. Patrones transversales Chronos	42

Capítulo 1

Introducción general

Este capítulo presenta una introducción al problema de la predicción de series de tiempo en el dominio financiero y su aplicación concreta en el desarrollo de un sistema de trading para el mercado de criptomonedas. Se describen el contexto y la motivación que impulsan la elección de la arquitectura Transformer como herramienta principal para el análisis. Además, se detalla la problemática del trading algorítmico, se revisa el estado del arte en la materia, se exponen los desafíos inherentes al manejo de datos financieros y, finalmente, se definen con claridad los objetivos y el alcance de este trabajo.

1.1. Motivación

La elección de este trabajo se fundamenta en la confluencia de dos pilares de gran relevancia actual: la complejidad del problema a resolver y la sofisticación de las herramientas seleccionadas para abordarlo. El trading de activos financieros es una actividad que busca generar rentabilidad a partir de las fluctuaciones de los precios en el mercado [1], lo que representa un desafío intelectual y técnico de alta complejidad debido a la naturaleza inherentemente ruidosa y dinámica de los mercados.

Gran parte de las decisiones en este campo se apoyan en el análisis técnico, que consiste en la interpretación de un amplio abanico de indicadores derivados de la actividad histórica de precios y volúmenes [2]. La combinación manual de estos indicadores para tomar decisiones de compra o venta requiere una experiencia considerable y una intuición desarrollada a lo largo de años. Además, este enfoque manual es difícil de escalar para analizar de manera simultánea la vasta cantidad de información disponible, lo que hace atractiva la idea de desarrollar un modelo de machine learning que pueda procesar y ponderar un gran número de indicadores técnicos para optimizar las decisiones de manera sistemática y objetiva.

Por otro lado, la arquitectura Transformer, gracias a su innovador mecanismo de autoatención, ha demostrado una capacidad excepcional para modelar dependencias a largo plazo en datos secuenciales y se ha consolidado como el estado del arte en campos como el procesamiento de lenguaje natural. Esta cualidad la convierte en una candidata ideal para el análisis de series de tiempo financieras [3], en el que eventos pasados, incluso lejanos en el tiempo, pueden tener una influencia significativa en el comportamiento futuro de los precios. A diferencia de los modelos recurrentes tradicionales, como las redes LSTM, que pueden tener dificultades para mantener información a través de secuencias muy largas, el

Transformer puede teóricamente capturar relaciones entre puntos distantes en el tiempo de manera más efectiva. Esto permitiría, por ejemplo, identificar patrones complejos que ayuden a evitar operaciones que parecen rentables a corto plazo pero que conllevan un alto riesgo, o, por el contrario, aprovechar oportunidades de baja rentabilidad pero con un riesgo casi nulo.

1.2. Descripción de la problemática

En la comercialización de activos financieros, el objetivo fundamental es generar una ganancia mediante la compra a un precio bajo y la venta a un precio más alto. Este principio, aunque simple en apariencia, sería trivial si se pudiera predecir el futuro de los precios con certeza. Sin embargo, la realidad de los mercados es que son sistemas inherentemente estocásticos y no lineales [4], como se puede apreciar en la volatilidad del precio de Bitcoin en la figura 1.1. Las personas que operan en estos mercados pueden clasificarse a grandes rasgos en dos categorías: los analistas fundamentales, que basan sus decisiones en el valor intrínseco de los activos y factores macroeconómicos, y los analistas técnicos, que se centran en el estudio de patrones históricos de precios y volúmenes para predecir movimientos futuros [5].



FIGURA 1.1. Evolución del precio del par BTC/USDT en el período analizado.¹

Este trabajo se enmarca estrictamente en el dominio del análisis técnico, con el propósito de desarrollar un sistema de trading algorítmico diseñado para operar de manera autónoma en el mercado spot de criptomonedas. Específicamente, se trabajó con el par BTC/USDT, uno de los más líquidos y volátiles del mercado. El objetivo del sistema es decidir, en intervalos de tiempo discretos, si es más conveniente mantener la posición en Bitcoin para aprovechar un posible aumento de su valor frente al Tether. Por otro lado, se busca determinar si es preferible conservar la posición en USDT para proteger el capital ante una eventual caída en el precio de la criptomoneda. Esta decisión binaria representa un problema de clasificación que es ideal para ser abordado con técnicas de deep learning.

¹Gráfico obtenido de la plataforma de trading Binance.

1.3. Estado del arte

El uso de inteligencia artificial para el trading ha ganado un protagonismo creciente en las últimas dos décadas y ha evolucionado desde modelos estadísticos tradicionales hasta complejas arquitecturas de redes neuronales [6]. Tanto en el ámbito comercial como en el académico, se ha reconocido el potencial de la IA para automatizar y mejorar la toma de decisiones. Empresas como Finbrain Technologies [7] ya ofrecen servicios de trading automático basados en señales predictivas generadas por modelos de IA, dirigidos tanto a inversores minoristas como institucionales.

En paralelo, el ámbito académico ha explorado extensamente el uso de machine learning para la optimización de portafolios y la predicción de mercados [8]. Los primeros enfoques se basaron en modelos como máquinas de soporte vectorial (SVM) y bosques aleatorios (random forests), que demostraron cierta capacidad predictiva. Con la llegada del deep learning, las redes neuronales recurrentes (RNN) y, en particular, las redes con memoria a corto y largo plazo (LSTM), se convirtieron en el estándar para el análisis de series de tiempo. Sin embargo, en los últimos años, la arquitectura Transformer ha emergido como una alternativa superior en muchos dominios secuenciales.

A pesar de los avances, muchos de estos enfoques se enfrentan a un obstáculo teórico fundamental: la Hipótesis de los Mercados Eficientes (EMH) [5]. Esta hipótesis, en su forma débil, sugiere que toda la información pasada ya está reflejada en el precio actual de los activos, lo que haría inútil cualquier intento de obtener ganancias extraordinarias si se basan únicamente en el análisis técnico. Aunque la validez de la EMH para el mercado de criptomonedas es un tema de intenso debate, se observa que la eficiencia del mercado tiende a aumentar a medida que este madura, lo que representa un desafío constante para el desarrollo de nuevos modelos predictivos.

1.4. El desafío de los datos financieros

La aplicación de técnicas de machine learning a los mercados financieros presenta un conjunto de desafíos únicos que no se encuentran comúnmente en otros dominios:

- Bajo ratio señal ruido: los movimientos de precios de los activos son inherentemente ruidosos. La señal predictiva, que representa la información útil para la predicción, a menudo se encuentra oculta bajo una gran cantidad de fluctuaciones aleatorias o “ruido”. Esto incrementa significativamente el riesgo de sobreajuste (overfitting), y el modelo aprende a memorizar el ruido de los datos de entrenamiento en lugar de la señal subyacente, lo que resulta en un bajo rendimiento en datos no vistos [9].
- No estacionariedad: las propiedades estadísticas de las series de tiempo financieras, como la media y la varianza, cambian con el tiempo. Esto se debe a que los mercados están influenciados por factores externos cambiantes, como cambios en la regulación económica o avances tecnológicos. Como resultado, los patrones aprendidos en un período de tiempo pueden no ser válidos en el futuro, lo que degrada el rendimiento del modelo a lo largo del tiempo [9].

- Costos de transacción: en un entorno de trading real, cada operación de compra o venta incurre en costos, como comisiones de la plataforma y deslizamientos (slippage). operar con alta frecuencia, aunque teóricamente podría permitir capturar más oportunidades de mercado, puede llevar a que las ganancias generadas por el modelo sean completamente consumidas por los costos transaccionales, lo que resulta en una pérdida neta [9].

Estos factores subrayan la necesidad de un diseño metodológico riguroso y específico para el dominio financiero, que vaya más allá de la simple aplicación de algoritmos estándar y considere estas particularidades.

1.5. Objetivos y alcance

El objetivo principal de este trabajo es estudiar y evaluar rigurosamente el desempeño de la arquitectura Transformer en la predicción de series de tiempo para su aplicación en un sistema de trading de criptomonedas en el mercado spot de la plataforma Binance.

Para alcanzar este objetivo principal, se han definido los siguientes objetivos específicos:

1. Realizar una revisión exhaustiva del estado del arte sobre la aplicación de modelos de deep learning (y en particular la arquitectura Transformer y sus variantes) a la predicción de series de tiempo financieras.
2. Implementar un modelo Transformer que utilice un conjunto de indicadores técnicos como datos de entrada para estimar si es óptimo posicionarse en BTC o en USDT en el par BTC/USDT durante el siguiente intervalo de tiempo.
3. Comparar el desempeño del modelo propuesto con una estrategia de referencia pasiva de “comprar y esperar” (buy and hold). El objetivo final no es maximizar la cantidad de USDT acumulado, sino la cantidad de BTC, lo que implica una estrategia de gestión de riesgo a largo plazo.
4. Probar distintas técnicas y configuraciones del modelo, como diferentes variantes de la arquitectura Transformer, distintos conjuntos de características de entrada y optimizaciones de hiperparámetros, para decidir el modelo final con el mejor desempeño.

El alcance de este trabajo se centra en la implementación y evaluación de un sistema de trading algorítmico, cuya arquitectura conceptual se ilustra en la figura 1.2. Este sistema incluye su operación simulada en un entorno de backtesting robusto [9], que utiliza datos históricos fuera de la muestra para evitar el sesgo de anticipación (*lookahead bias*), así como la capacidad de operar en tiempo real a través de la API de Binance. Adicionalmente, se buscará determinar una frecuencia de operación que optimice el equilibrio entre la disponibilidad de datos para el entrenamiento del modelo y el impacto de los costos transaccionales en la rentabilidad final.

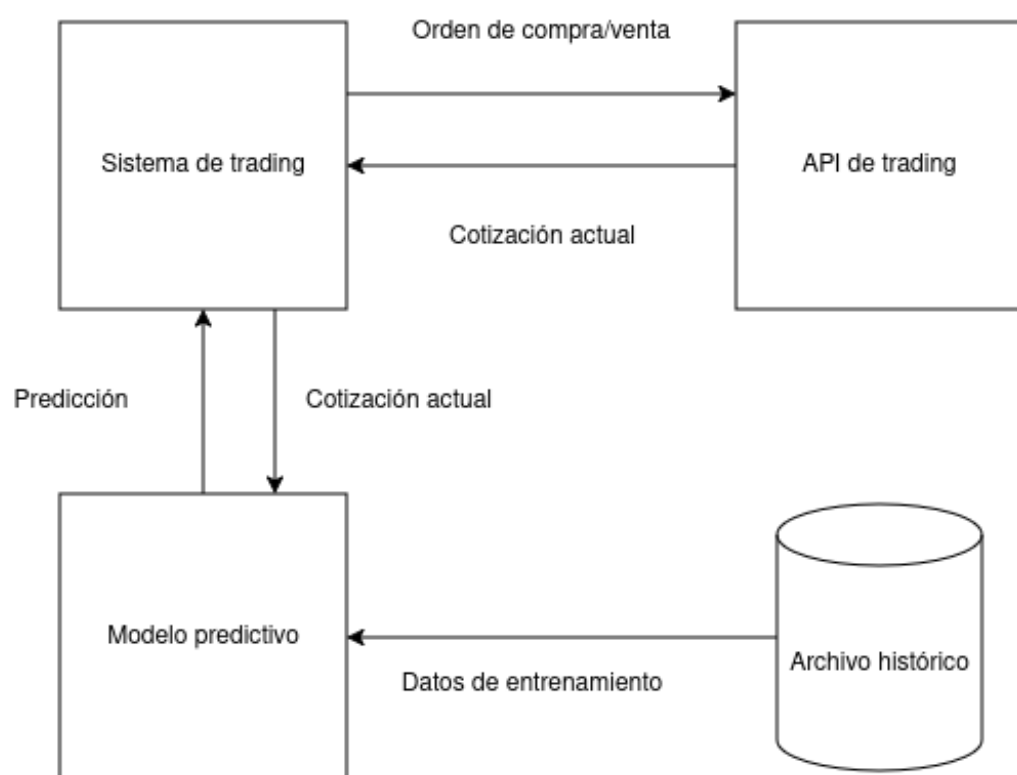


FIGURA 1.2. Diagrama conceptual de la arquitectura del sistema de trading.

Capítulo 2

Introducción específica

En este capítulo se cubren aspectos fundamentales de trading, análisis técnico y conceptos clave de aprendizaje automático, indispensables para comprender el desarrollo y los resultados del trabajo.

2.1. Trading y análisis técnico

El trading es la compra y venta de activos financieros a corto plazo para obtener ganancias, diferenciándose de la inversión a largo plazo. Los traders se clasifican por la duración de las operaciones (scalping, diario, swing, eventos) y por el tipo de análisis (fundamental o técnico) que emplean [1, 10, 11].

El análisis técnico pronostica movimientos de precios futuros a través del estudio de datos históricos del mercado, como precio y volumen [2, 12, 13]. Se basa en la Teoría de Dow [14], que postula:

- El precio lo descuenta todo.
- Existen tres tipos de tendencias (primaria, secundaria y terciaria) con fases distintivas.
- Las tendencias deben ser confirmadas por el volumen y la alineación de diferentes índices.
- Las tendencias se mantienen vigentes hasta que haya una señal clara de reversión.

2.2. Aprendizaje supervisado

El aprendizaje automático (machine learning) se clasifica comúnmente en aprendizaje supervisado, no supervisado y por refuerzo. El aprendizaje supervisado se basa en el uso de datos etiquetados para entrenar un modelo que aprenda a predecir una salida a partir de una entrada [15].

Esta técnica se divide en dos categorías principales:

- Clasificación: el modelo predice una categoría discreta o una etiqueta. Por ejemplo, predecir si un activo subirá o bajará en el periodo siguiente.
- Regresión: el modelo predice un valor numérico continuo. Por ejemplo, predecir el precio futuro de un activo financiero.

Para este trabajo, se optó por un enfoque de clasificación, en el que el modelo predice la dirección del movimiento del precio (subida o bajada) en lugar de la magnitud exacta.

En el ámbito del aprendizaje automático, los ensambles son el resultado de combinar múltiples modelos entrenados individualmente para obtener un desempeño general superior al que se lograría con un solo modelo [16]. La idea es que los puntos débiles de un modelo puedan ser compensados por los puntos fuertes de otros, lo que mejora la robustez y la precisión de las predicciones. Existen diferentes técnicas de ensamble, como bagging y boosting, que combinan los modelos de distintas maneras [16].

2.3. Optimización de hiperparámetros

Es fundamental distinguir entre los parámetros de un modelo y sus hiperparámetros. Los parámetros son valores aprendidos por el modelo durante el entrenamiento a partir de los datos (por ejemplo, los pesos en una red neuronal o la pendiente en una regresión lineal). Los hiperparámetros, en cambio, son configuraciones que se definen antes del entrenamiento y no se modifican durante este proceso (por ejemplo, la tasa de aprendizaje o la profundidad de un árbol) [15].

El rendimiento de un modelo depende tanto de sus parámetros como de sus hiperparámetros. La optimización de hiperparámetros es el proceso de buscar la combinación de valores de hiperparámetros que resulta en el mejor desempeño del modelo. Esto a menudo implica entrenar múltiples modelos con diferentes combinaciones y seleccionar el que ofrezca los mejores resultados en un conjunto de validación [15]. Aunque la optimización de hiperparámetros es conveniente, puede ser computacionalmente intensiva para modelos complejos, como las redes neuronales profundas o los modelos basados en aprendizaje por refuerzo.

2.4. Transformers

La arquitectura Transformer, introducida por [17], revolucionó el procesamiento de secuencias y ha demostrado ser altamente efectiva en el análisis de series de tiempo. A diferencia de los modelos recurrentes, los Transformers se basan en mecanismos de autoatención (self-attention) para modelar dependencias de largo alcance de manera eficiente.

Recientemente, han surgido modelos fundacionales preentrenados que adaptan arquitecturas de lenguaje para el pronóstico de series temporales. Entre ellos destaca Chronos [18], un marco de trabajo que trata las series temporales como un lenguaje. El núcleo de Chronos es su proceso de tokenización, que convierte una serie de valores reales en una secuencia de tokens discretos mediante dos pasos:

- Escalado (Scaling): se normaliza la serie para adecuarla a un rango apropiado para la cuantización.
- Cuantización (Quantization): la serie escalada se convierte en tokens discretos mediante una cuantización uniforme (uniform binning), donde los valores se asignan a uno de B contenedores (bins) predefinidos.

Una vez tokenizada la serie, se utiliza una arquitectura Transformer estándar (por ejemplo, T5) que se entrena con un objetivo de lenguaje, como predecir el siguiente token, usando una función de pérdida de entropía cruzada. Este enfoque, conocido como regresión por clasificación, permite al modelo aprender patrones temporales complejos sin necesidad de componentes específicos para series de tiempo, como capas de descomposición de tendencia o estacionalidad [18].

Siguiendo la figura 2.1 se ve la fase de tokenización (izquierda), la serie temporal de entrada se somete a un proceso de escalado y cuantización para transformarla en una secuencia de tokens. Durante el entrenamiento (centro), estos tokens se introducen en un modelo de lenguaje, ya sea de arquitectura codificador-decodificador o solo decodificador, el cual se optimiza mediante la pérdida de entropía cruzada. Finalmente, en la etapa de inferencia (derecha), se muestrean tokens de forma autorregresiva para volver a convertirlos en valores numéricos; este proceso se repite para generar múltiples trayectorias que permiten obtener una distribución predictiva completa.

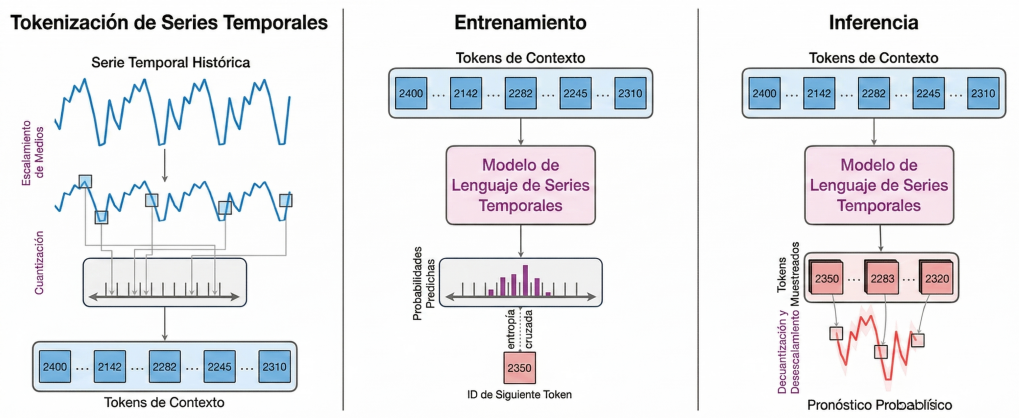


FIGURA 2.1. Representación de alto nivel de Chronos. Fase de tokenización (izquierda), entrenamiento (centro), e inferencia (derecha).

2.5. Metodologías de evaluación y gestión de riesgo

Para garantizar la robustez y viabilidad de una estrategia de trading, es fundamental contar con un marco sólido de evaluación y herramientas para la gestión del riesgo. En esta sección se detallan las metodologías y métricas clave que se utilizarán para validar el desempeño de los modelos y para tomar decisiones informadas sobre la asignación de capital.

2.5.1. Técnicas de selección de características

La selección de características es un paso crucial para reducir la dimensionalidad, mejorar el rendimiento del modelo y evitar el sobreajuste. Se utilizaron diversas métricas para evaluar la importancia de las características generadas:

- Disminución media de impureza (MDI): específica de modelos basados en árboles como Random Forest, esta métrica mide la importancia de una característica por cuánto reduce la impureza (por ejemplo, impureza de Gini) en los nodos del árbol.

- Disminución media de exactitud (MDA): se evalúa la importancia de una característica con una permuta aleatoria de sus valores y se mide la caída resultante en el rendimiento del modelo. Una caída significativa indica que la característica es importante.
- Importancia de característica individual (SFI): entrena y evalúa un modelo con una sola característica a la vez para medir su poder predictivo de forma aislada.
- Análisis de componentes principales (PCA): aunque es una técnica de reducción de dimensionalidad, se utilizó para transformar características correlacionadas en un conjunto de componentes ortogonales, cuya importancia se evaluó posteriormente con las métricas mencionadas.

2.5.2. Técnicas de etiquetado

El etiquetado define el objetivo que el modelo de aprendizaje supervisado debe predecir. Se exploraron las siguientes técnicas:

- Etiquetado por horizonte fijo: asigna etiquetas (por ejemplo, “sube” o “baja”) según el movimiento del precio después de un intervalo de tiempo fijo. Su principal desventaja es que no se adapta a la volatilidad del mercado.
- Etiquetado dinámico: similar al anterior, pero los umbrales para definir las etiquetas se ajustan en función de la volatilidad reciente del mercado.
- Etiquetado con barrera triple (triple-barrier method): este método, popularizado por López de Prado [9], establece tres barreras para cada observación: una barrera superior (objetivo de ganancia), una inferior (límite de pérdida) y una barrera de tiempo vertical (expiración de la operación). La primera barrera que el precio toca determina la etiqueta. Este enfoque es más robusto porque se adapta dinámicamente a la volatilidad del mercado.

2.5.3. Backtesting

El backtesting es el pilar fundamental para evaluar una estrategia de trading. Consiste en simular el comportamiento de la estrategia al utilizar datos históricos para estimar cómo se habría desempeñado en el pasado. Un backtesting riguroso debe evitar el sesgo de retrospcción (look ahead bias) y asegurar que las decisiones del modelo en un punto del tiempo se basen únicamente en información disponible hasta ese momento [9].

2.5.4. Validación cruzada

La validación cruzada (Cross Validation) es una técnica para evaluar la capacidad de generalización de un modelo y prevenir el sobreajuste. Sin embargo, la validación cruzada estándar (como K-Fold) no es adecuada para series de tiempo debido a la dependencia temporal de los datos. En su lugar, se utilizan métodos especializados. Uno de ellos es el Purged K-Fold cross-validation. Esta técnica introduce dos modificaciones clave. Primero, se inserta un embargo o período de exclusión después de cada conjunto de validación para evitar que la información futura se filtre al conjunto de entrenamiento. Segundo, purgan del conjunto de entrenamiento aquellas observaciones cuya etiqueta podría estar influenciada por datos del conjunto de validación [9].

2.5.5. Dimensionamiento de la apuesta

El dimensionamiento de la apuesta (Bet sizing) se refiere al proceso de determinar la cantidad de capital que se arriesgará en una operación [9]. Un modelo puede predecir la dirección del precio (clasificación) o la probabilidad de un movimiento (regresión), pero el bet sizing traduce esa predicción en una decisión de inversión concreta. El objetivo es maximizar los rendimientos ajustados por riesgo. Un método común es utilizar una función sigmoidea para convertir las probabilidades de predicción del modelo en un tamaño de apuesta, lo que permite al sistema invertir con más confianza cuando el modelo está más seguro.

2.5.6. Ratio de Sharpe y sus variantes

El ratio de Sharpe [19] es una métrica clave utilizada para evaluar el rendimiento de una inversión ajustado por el riesgo. Este ratio ayuda a determinar la rentabilidad de una inversión en exceso de la tasa libre de riesgo, por unidad de desviación estándar del rendimiento. Cuanto mayor sea el ratio de Sharpe, más atractivo es el rendimiento ajustado por riesgo de la estrategia. La fórmula general se ve en 2.1.

$$S = \frac{R_p - R_f}{\sigma_p} \quad (2.1)$$

donde:

- R_p : es el rendimiento de la cartera.
- R_f : es la tasa de rendimiento de un activo libre de riesgo.
- σ_p : es la desviación estándar de la rentabilidad de la cartera (medida del riesgo).

Es importante destacar que el ratio de Sharpe tradicional asume que los retornos se distribuyen normalmente (como una campana de Gauss), una premisa que rara vez se cumple en los mercados financieros, en los que los retornos suelen presentar asimetría y colas pesadas. Para una evaluación más realista del riesgo, López de Prado introduce métricas ajustadas que abordan estas limitaciones [9].

Existen variantes de este ratio que abordan ciertas limitaciones, como el Sortino ratio [20], que solo considera la volatilidad a la baja (riesgo de cola), o el Calmar ratio [21], que utiliza la máxima caída (max drawdown) como medida de riesgo en lugar de la desviación estándar. Estas variantes son útiles para obtener una perspectiva más completa del perfil de riesgo rendimiento de una estrategia de trading, especialmente en mercados volátiles como el de las criptomonedas.

Para una evaluación aún más rigurosa y para combatir el riesgo de sobreajuste en el backtesting, se pueden considerar las siguientes métricas avanzadas propuestas por López de Prado:

- Probabilidad de sobreajuste de backtest (PBO): mide la probabilidad de que la estrategia seleccionada como “la mejor” dentro de la muestra de entrenamiento rinda por debajo de la mediana en datos fuera de la muestra, simplemente debido al sesgo de selección.

- *Probabilistic sharpe ratio* (PSR): corrige el ratio de Sharpe tradicional y lo penaliza si los retornos tienen sesgo negativo (skewness), colas pesadas (kurtosis) o si el historial de datos es demasiado corto. Proporciona la probabilidad real de que el Sharpe de la estrategia sea mayor que un valor de referencia (benchmark).
- *Deflated sharpe ratio* (DSR): va un paso más allá del PSR y ajusta el umbral de aceptación a partir del número de ensayos independientes (trials) y la varianza de esos ensayos. Si se probaron numerosas variaciones de una estrategia, el DSR exigirá un ratio de Sharpe significativamente más alto para considerar que la estrategia es estadísticamente significativa.

Capítulo 3

Diseño e implementación

En este capítulo se explica el desarrollo de la solución alcanzada. Se describen los datos que se procesaron, las estrategias que se probaron, así como el criterio que se usó para compararlas. También se agregan detalles sobre el diseño de la implementación del bot de trading.

3.1. Conjunto de datos y preprocesamiento

Esta sección describe el conjunto de datos utilizado y las estrategias de preprocesamiento aplicadas para el desarrollo del trabajo. Se detallan las consideraciones sobre la frecuencia de los datos, las técnicas de construcción de barras y el análisis de la importancia de las características.

3.1.1. Conjunto de datos

El trabajo se basó en el análisis de datos del mercado spot de Bitcoin. El mercado spot se define como aquel en el que los activos financieros, en este caso Bitcoin (BTC) frente al Tether (USDT), se compran y venden para su liquidación y entrega inmediata. Las variables disponibles para este mercado incluyen el precio de apertura, el precio de cierre, el valor mínimo y máximo alcanzado, el volumen comercializado en unidades de Bitcoin y en USDT, y la hora exacta en la que se tomó cada medición.

Estos datos se recopilaron desde el 1 de enero de 2020 a través de la plataforma de Binance. Inicialmente, se experimentó con una frecuencia de una hora, la cual presentó la desventaja de acotar el volumen de datos disponibles para el entrenamiento del modelo. Además, se observó que las estrategias probadas con esta frecuencia arrojaban peores resultados. Se infirió que esto se debía, posiblemente, a la menor cantidad de datos disponibles, lo que limitó la capacidad del modelo para aprender patrones robustos. Por lo anterior, se optó por utilizar una frecuencia de un minuto, lo que generó una mejora significativa en el desempeño de las estrategias estudiadas al proporcionar un conjunto de datos más denso y amplio.

3.1.2. Estrategias de preprocesamiento

A continuación, se detalla el preprocesamiento realizado, común a la mayoría de los modelos explorados y basado en las recomendaciones de Marcos López de Prado para finanzas cuantitativas.

Para asegurar la estacionariedad de las series, se aplica la diferenciación fraccional, una técnica avanzada recomendada en la literatura explorada. Dado que las

series de precios financieras no son estacionarias, esta técnica busca un orden mínimo de diferenciación (d), que puede ser un número fraccionario (por ejemplo, $d=0.4$), para hacer la serie estacionaria (según la prueba de Dickey-Fuller aumentada) sin eliminar por completo la información de la serie original. Esto contrasta con la diferenciación clásica ($d=1$) que puede borrar demasiada “memoria”.

Para el preprocesamiento de los datos, se aplicaron diversas estrategias con el fin de optimizar su calidad y relevancia para el modelo:

- Conversión a micro Bitcoins (uBTC): se realizó una conversión de BTC a uBTC, una práctica estándar en el análisis de criptomonedas para manejar unidades más pequeñas y facilitar los cálculos.
- Agrupación por volumen: se exploraron formas de agrupar los datos no por intervalos de tiempo fijos, sino por volumen en unidades constantes tanto de BTC como de USDT. Adicionalmente, se probaron las *volume imbalance bars* y las *tick imbalance bars*. Las *volume imbalance bars* se construyen cuando se alcanza un volumen neto desequilibrado entre compras y ventas, y las *tick imbalance bars* se forman con un desequilibrio neto en el número de ticks (cambios de precio). Después de una investigación exhaustiva de la literatura y pruebas empíricas, se decidió continuar con la agrupación de barras por volumen fijo. Esta elección se fundamentó en que las barras de volumen aportan una mejora notable en las propiedades estadísticas de la distribución del precio de Bitcoin, ya que reducen la autocorrelación y la heterocedasticidad, lo que las hace más adecuadas para el entrenamiento de modelos de machine learning. Se adoptó un límite de volumen de 50.000 uBTC, que proporcionó un equilibrio satisfactorio entre la cantidad de puntos de datos obtenidos y la reducción de la varianza correspondiente a cada barra.

Finalmente, antes del entrenamiento, se aplican transformaciones estándar como el escalado y la reducción de dimensionalidad. El escalado, específicamente *StandardScaler*, estandariza las características a una media de 0 y varianza de 1, crucial para modelos sensibles a la escala (como SVM o redes neuronales), lo que asegura que el escalador se ajuste únicamente en los datos de entrenamiento para evitar fugas de información. Opcionalmente, mediante un hiperparámetro, se utiliza el análisis de componentes principales (PCA) para reducir la dimensionalidad y ortogonalizar las características, lo que puede beneficiar a algunos modelos y mejorar la interpretabilidad.

En síntesis, el preprocesamiento representa un pipeline sofisticado que transforma los datos brutos de precios en un formato estructurado, estacionario y etiquetado de manera robusta, listo para ser utilizado por los modelos de aprendizaje de máquina. La figura 3.1 ilustra este proceso.

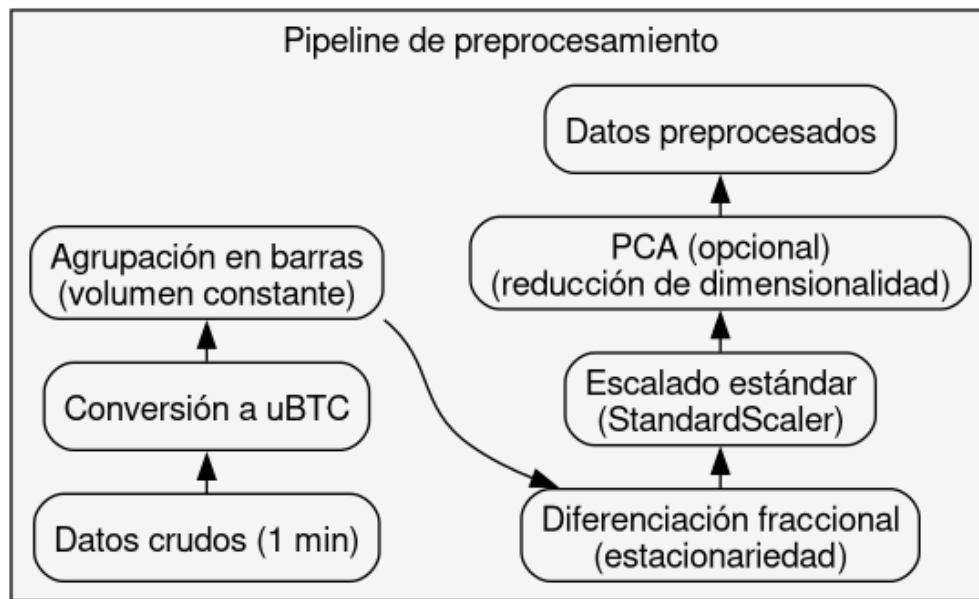


FIGURA 3.1. Diagrama del pipeline de preprocesamiento de datos.

3.1.3. Ingeniería de características y análisis de importancia

Una vez estructuradas las barras, se procede con la ingeniería de características, donde se construye un conjunto extenso de atributos para los modelos. Esto incluye indicadores técnicos.

Para la ingeniería de características, se generó una extensa lista de indicadores técnicos clásicos como el RSI, el Oscilador Asombroso (AO), Williams %R y ROC para momentum; MACD, ADX, Aroon, CCI y Vortex para tendencia; y las Bandas de Bollinger, Canales de Keltner y Canales de Donchian para volatilidad y fluctuación. Adicionalmente, se incorporan características basadas en retornos porcentuales (`pct_change`) y sus retardos. En el pipeline específico de Chronos, el modelo se utiliza para generar embeddings a partir de ventanas de la serie de precios. Estos embeddings, que encapsulan patrones complejos, sirven como características adicionales para un modelo tabular, como XGBoost. Adicionalmente, se hizo uso de técnicas de reducción de dimensionalidad como el análisis de componentes principales (PCA). Para determinar las características más relevantes para el modelo final, se realizó un análisis de importancia en el cual se utilizó un conjunto de métricas, entre ellas: la disminución media de impureza (MDI), la disminución media de exactitud (MDA) y la importancia de característica individual (SFI). También se analizaron las funciones de autocorrelación (ACF) y autocorrelación parcial (PACF) de los componentes principales para identificar dependencias temporales.

3.2. Estrategias de referencia

Para establecer un punto de comparación robusto, se tomó como principal referencia el trabajo de Guilherme Palazzo [22]. Su metodología destaca por la aplicación rigurosa de técnicas avanzadas de machine learning financiero, y varios de

sus componentes clave fueron adoptados en este trabajo para construir una base sólida sobre la cual evaluar la arquitectura Transformer propuesta.

Los elementos metodológicos que se incorporaron de dicho trabajo son los siguientes:

- Muestreo basado en volumen: se adoptó la técnica de agrupar los datos en barras de volumen constante en lugar de intervalos de tiempo fijos. Este enfoque mitiga problemas comunes en las series de tiempo financieras, como la heterocedasticidad, y mejora la calidad de los datos para el entrenamiento de modelos.
- Formulación como clasificación: al igual que en el trabajo de referencia, el problema se formuló como una tarea de clasificación para predecir la dirección del movimiento del precio (alza o baja), en lugar de intentar predecir el valor exacto del precio.
- Inspiración en el etiquetado dinámico: la estrategia de etiquetado se inspiró en el enfoque de Palazzo de no utilizar horizontes de tiempo fijos. En este trabajo, dicha idea se implementó a través del método de la barrera triple, que se adapta dinámicamente a la volatilidad del mercado.
- Validación de modelos basados en árboles: el éxito de XGBoost en el trabajo de referencia sirvió como validación para su uso como un componente eficaz en la arquitectura final, en conjunto con los embeddings generados por Chronos.

Al integrar estos componentes, se asegura que la evaluación del rendimiento del modelo Transformer se realice contra una estrategia de referencia competitiva y bien fundamentada en el estado del arte.

3.3. Arquitectura transformer propuesta

Para la arquitectura principal de este trabajo, se utilizó el modelo fundacional Chronos, cuya conceptualización fue detallada en el capítulo 2. En lugar de utilizar el modelo para pronóstico directo, se aprovechó su capacidad para generar representaciones ricas de las series de tiempo.

El enfoque implementado fue el siguiente:

1. Se tomaron las series de tiempo de los indicadores técnicos seleccionados.
2. Cada serie se procesó con un modelo Chronos pre entrenado para extraer los embeddings de sus capas ocultas. Estos embeddings son vectores de alta dimensionalidad que encapsulan la información contextual y los patrones temporales aprendidos por el Transformer.
3. Los embeddings resultantes de todos los indicadores se concatenaron para formar un único vector de características.
4. Este vector de características se utilizó como entrada para un modelo tabular más tradicional y computacionalmente eficiente, en este caso, XGBoost o LightGBM, que fue el encargado de realizar la tarea de clasificación final (predecir si el precio subirá o bajará).

Esta estrategia de dos pasos permite combinar el poder de los modelos fundacionales para la extracción de características complejas con la eficiencia y robustez de los modelos basados en árboles para la clasificación en datos tabulares. Finalmente, se experimentó con diferentes tamaños del modelo Chronos (small, base y large) para analizar el compromiso entre el rendimiento predictivo y el costo computacional.

3.4. Estrategia de etiquetado

Para definir el objetivo de predicción del modelo, se evaluaron varias técnicas de etiquetado. Tras analizar los métodos de horizonte fijo y etiquetado dinámico, se adoptó el método de la barrera triple (Triple-Barrier Method). Esta técnica, recomendada en la literatura para mercados financieros, ofrece un enfoque más adaptativo y robusto al definir las etiquetas en función de la volatilidad del activo, con barreras para ganancias, pérdidas y tiempo de expiración. Para etiquetar un punto se puede elegir una etiqueta independiente para la barrera vertical o agruparla con las barreras horizontales según si el punto final es mayor o menor que el inicial, si es menor se agrupa con la etiqueta correspondiente a la barrera inferior y, si es mayor, con la superior. Esta alternativa permite mantener las clases a predecir balanceadas con lo cual fue la seleccionada.

En la figura 3.2 se ve cómo a partir de un punto de Incepción (inicio de la operación), se definen tres límites: una barrera de ganancia superior (púrpura) y una barrera de pérdida inferior (cian) que determinan si la operación se clasifica como positiva o negativa si el precio las toca, y una barrera de tiempo vertical (negra) que sirve de vencimiento, etiqueta la posición por su rendimiento hasta ese momento si ninguna barrera horizontal fue alcanzada previamente.

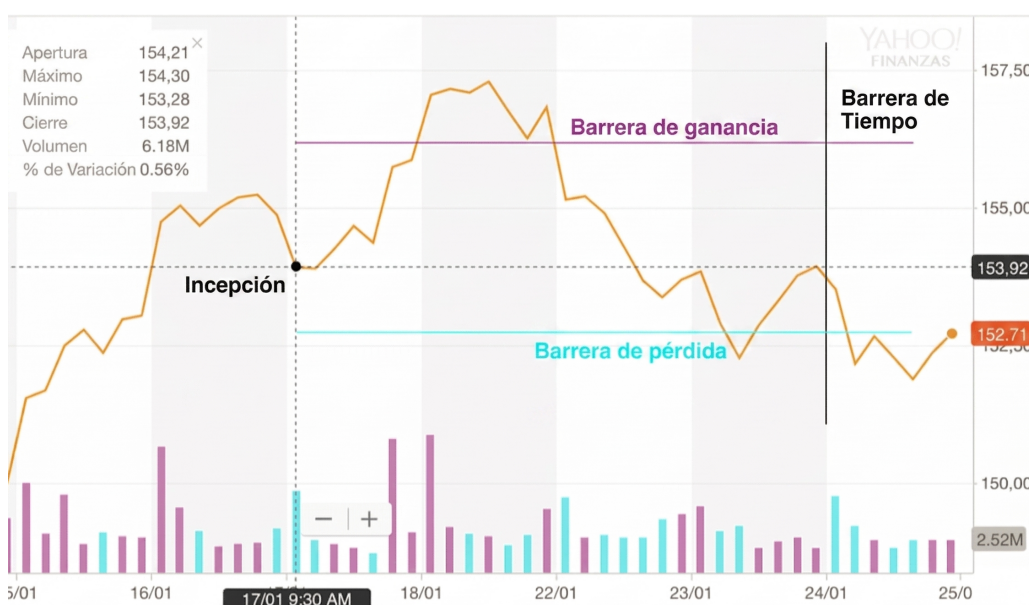


FIGURA 3.2. Aplicación del método de etiquetado de triple barrera sobre un gráfico de precios.

3.5. Framework de backtesting y evaluación

Para asegurar una evaluación rigurosa y libre de sesgos, se implementó un motor de backtesting a medida en Python. Esta decisión se tomó debido a la necesidad de utilizar metodologías avanzadas como la validación cruzada combinatoria con purga y embargo (*Combinatorially Purged Cross-Validation*). Este método es esencial para prevenir la fuga de información y el sobreajuste al evaluar modelos en series de tiempo financieras, lo que garantiza que el rendimiento medido sea una estimación realista del comportamiento del modelo en datos futuros.

3.5.1. Optimización de hiperparámetros: búsqueda bayesiana con Optuna

La optimización de hiperparámetros es una de las etapas más críticas en la construcción de un modelo de machine learning robusto, especialmente en el dominio financiero donde el riesgo de sobreajuste es muy alto. El enfoque adoptado en este proyecto se centró en la eficiencia y la rigurosidad estadística, y separa estrictamente la fase de optimización de la evaluación final.

Para evitar esto, la búsqueda de hiperparámetros se realizó únicamente con el conjunto de entrenamiento, y se validó el rendimiento de cada configuración mediante purged k-fold cross-validation. El conjunto de prueba final (hold-out set) se mantuvo completamente aislado y solo se utilizó una única vez al final de todo el proceso para evaluar el modelo definitivo.

Al tratarse de un problema de clasificación, y por el desbalance de clases inherente a las señales de trading, se utilizó la métrica F1 ponderada por clase para evaluar el desempeño. Esta métrica proporciona una medida balanceada entre la precisión y la exhaustividad del modelo, ajustada a la frecuencia de cada clase, lo que la hace más representativa que la exactitud simple.

Para llevar a cabo la búsqueda de manera eficiente, se utilizó la biblioteca Optuna [23]. En lugar de recurrir a métodos de fuerza bruta como la búsqueda en grilla (Grid Search) o a la ineficiente búsqueda aleatoria (Random Search), Optuna implementa un enfoque de optimización Bayesiana.

Cada prueba individual dentro del estudio de Optuna consistió en un proceso completo y computacionalmente intensivo:

1. Sugerencia de hiperparámetros: Optuna propone una nueva combinación de hiperparámetros (ej. `volume_threshold`, `tau`, `n_estimators`, etc.).
2. Ejecución del pipeline completo: con estos hiperparámetros, se ejecuta el pipeline desde cero sobre el conjunto de entrenamiento:
 - a) Se estructuran los datos en barras de volumen.
 - b) Se generan las características (features).
 - c) Se crean las etiquetas (labels) según el método definido.
 - d) Se ejecuta el proceso completo de purged k-fold cross-validation de n folds.
3. Cálculo de la métrica objetivo: se calcula el F1-Score ponderado para cada una de las validaciones cruzadas.

La figura 3.3 ilustra este proceso iterativo.

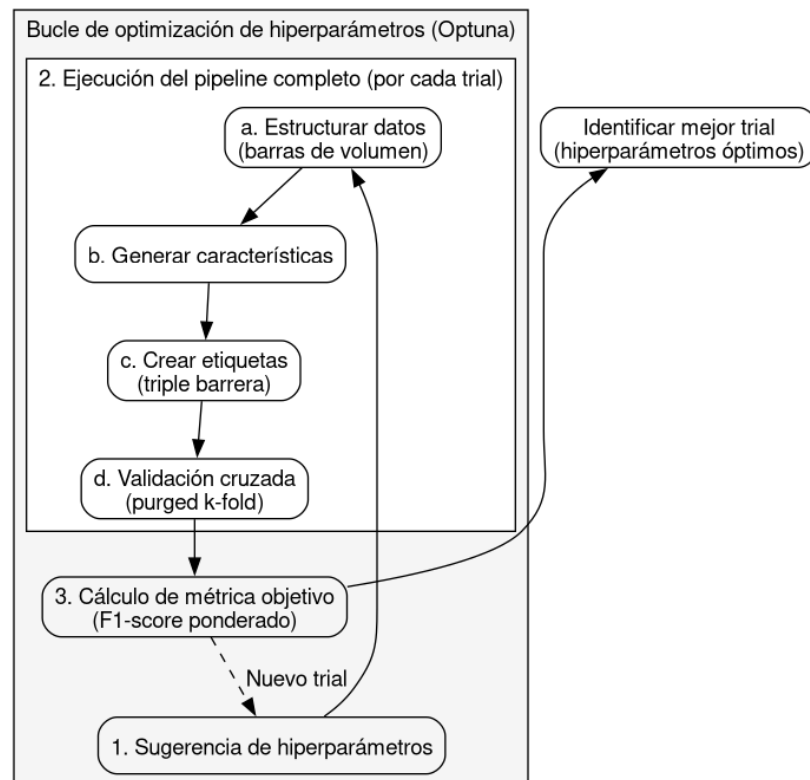


FIGURA 3.3. Diagrama del proceso de optimización de hiperparámetros con Optuna.

Este procedimiento asegura que la evaluación de cada conjunto de hiperparámetros fuera robusta y no dependiera de una única partición de datos.

Una vez que Optuna finaliza todas las pruebas, se identifica el “mejor trial”, es decir, la combinación de hiperparámetros que produjo el F1-Score promedio más alto. Estos hiperparámetros se consideraron óptimos.

Finalmente, se entrenó un único modelo con estos hiperparámetros óptimos sobre la totalidad del conjunto de entrenamiento. Este modelo final y definitivo fue el que se evaluó, por única vez, en el conjunto de prueba (hold-out) para obtener una estimación realista y sin sesgos de su rendimiento esperado en datos no vistos.

La evaluación de los modelos se centró exclusivamente en su rendimiento en el backtest, lo que separa estrictamente esta fase de la optimización de hiperparámetros. Dicha búsqueda se realizó una única vez durante la etapa de investigación y validación del modelo, con un conjunto de datos distinto al de la evaluación final.

3.6. Modelos explorados

Durante la fase de investigación, se exploró un amplio espectro de modelos para establecer un punto de referencia y comprender la complejidad del problema.

Inicialmente, se evaluaron estrategias de trading clásicas basadas en indicadores técnicos, como el cruce de medias móviles, las bandas de Bollinger, el MACD y

las divergencias en el RSI. Si bien estos métodos son populares, demostraron ser demasiado rígidos, poco adaptativos, y propensos al sobreajuste.

Posteriormente, se probaron modelos estadísticos tradicionales para series temporales, incluyendo ARIMA, SARIMA, y variantes más complejas como ARIMAX con GARCH como variable exógena y Prophet de Facebook. Estos modelos, aunque más sofisticados, lucharon por capturar las dinámicas no lineales y no estacionarias del precio de Bitcoin y no superaron a un desempeño aleatorio.

Luego, se exploraron modelos que sean *drop in replacements* al marco presentado por [22]. Para seguir con la recomendación de López de Prado sobre la idoneidad de los modelos basados en árboles tipo random forest para datos financieros. Además, se exploraron alternativas a XGBoost. Se utilizó la biblioteca AutoGluon, que permite entrenar y comparar de forma automática un conjunto de modelos tabulares de última generación, incluyendo diferentes configuraciones de redes neuronales y ensambles, para encontrar un posible reemplazo al modelo de Palazzo.

Finalmente, en cuanto a la arquitectura principal de Chronos, se experimentó con diferentes tamaños del modelo T5 (small, base y large) para analizar el compromiso entre el rendimiento predictivo y el costo computacional, se busca el equilibrio óptimo para una implementación práctica.

3.7. Diseño del bot

El bot de trading se diseñó con una arquitectura modular para facilitar su configuración, mantenimiento y extensibilidad. Se divide en cuatro componentes principales que operan de forma desacoplada:

1. Orquestador: es el punto de entrada de la aplicación. Su función es instanciar, configurar y conectar los demás componentes. Inicia el ciclo de operación del bot.
2. Motor del bot: contiene el bucle principal de ejecución. En cada iteración, orquesta el flujo de trabajo: solicita los datos más recientes del mercado al módulo de intercambio, pasa estos datos al módulo de estrategia para obtener una señal de trading y, si la señal lo indica, instruye al módulo de intercambio para que ejecute la orden correspondiente.
3. Módulo de exchange: abstrae toda la comunicación con la API del exchange de criptomonedas. Es responsable de obtener datos de mercado (precios, volúmenes) y de ejecutar órdenes (compra, venta). Su diseño intercambiable permite adaptar el bot a diferentes plataformas (por ejemplo, Binance, KuCoin) con mínimas modificaciones.
4. Módulo de estrategia: carga el modelo de machine learning pre-entrenado (en este caso, el ensamble de Chronos y XGBoost) y lo se utiliza para procesar los datos de mercado y generar una señal de trading: comprar, vender o mantener.

En esencia, el bot opera en un ciclo continuo: recolecta datos, predice una señal con el modelo y actúa sobre esa predicción. Esta separación de responsabilidades garantiza que la lógica de la estrategia de trading esté completamente aislada de

los detalles de implementación de la comunicación con el mercado. de la comunicación con el mercado. A continuación se proporciona una descripción y vista técnica y exhaustiva de cómo interactúan los cuatro componentes desacoplados (orquestador, motor, exchange y estrategia) a lo largo del ciclo de vida de la aplicación.

Fase de inicio y orquestación

Esta fase inicializa el sistema y demuestra el rol del orquestador como punto de entrada de la aplicación, encargándose de instanciar y conectar las piezas. Véase la figura 3.4, 3.5, y 3.6.

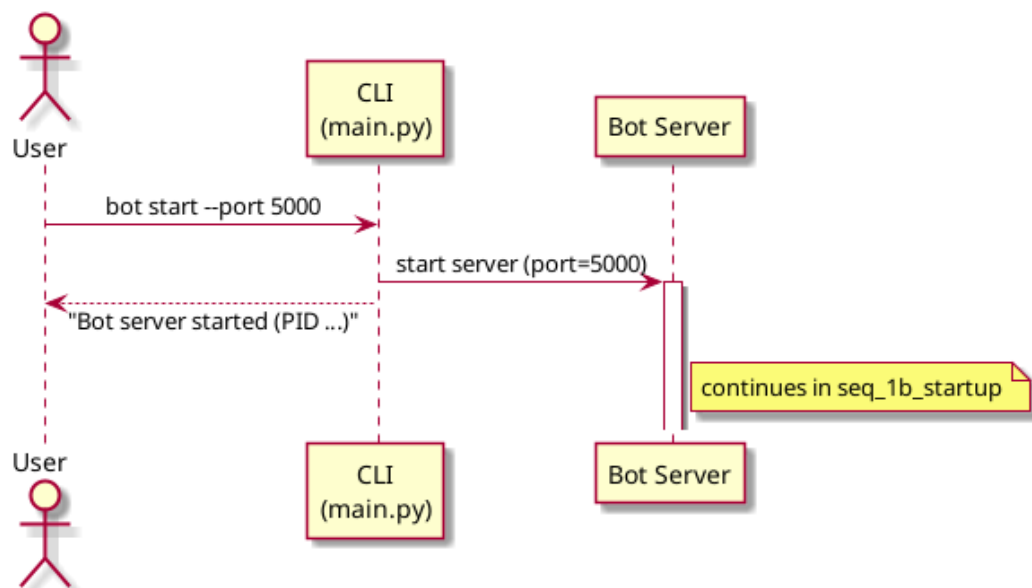


FIGURA 3.4. Diagrama del proceso de inicialización del bot.

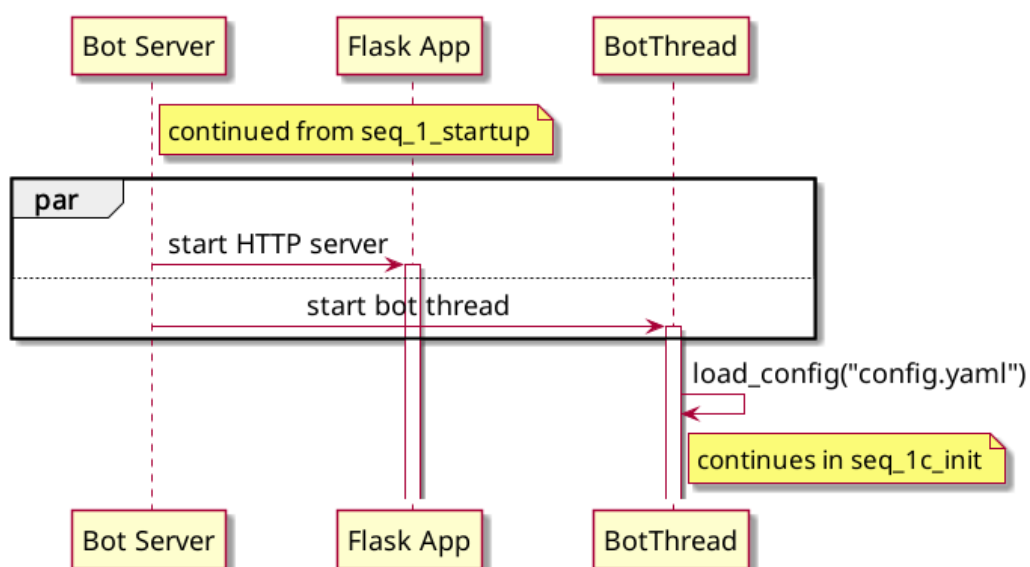


FIGURA 3.5. Diagrama del proceso de inicialización del bot.

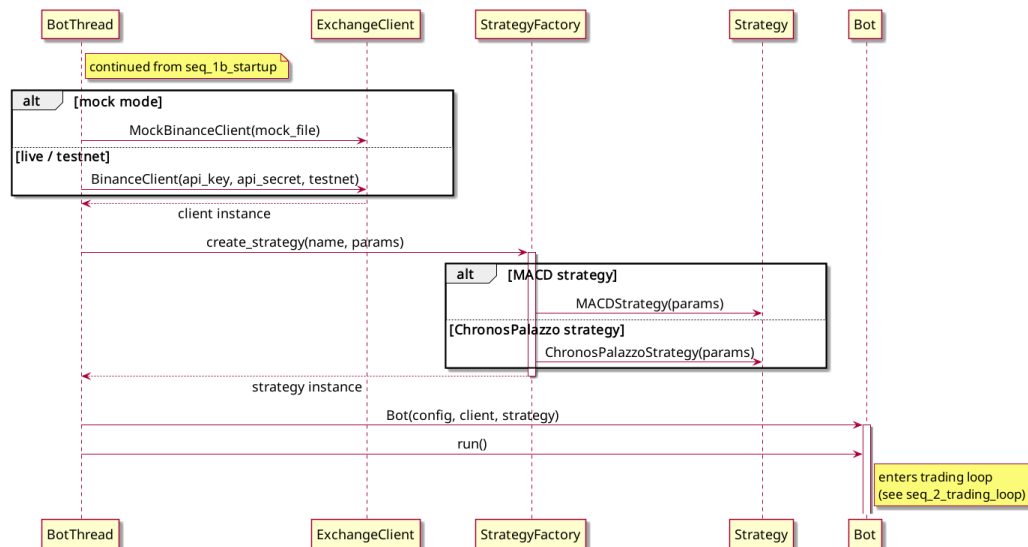


FIGURA 3.6. Diagrama del proceso de inicialización del bot.

A continuación se describen los pasos de la fase de inicialización:

- Con la interacción inicial del usuario: el proceso comienza cuando el usuario introduce el comando `bot start -port 5000` a través de la interfaz de línea de comandos (cli), alojada en `main.py`.
- Creación del servidor en segundo plano: la cli no ejecuta el bot de forma bloqueante. En su lugar, delega la tarea al módulo `subprocess` para levantar un servidor (`run_server`) en el puerto 5000.
- Confirmación de inicio: la cli devuelve inmediatamente un mensaje al usuario y confirma que el servidor ha iniciado y proporciona el identificador del proceso (pid). Así libera la terminal.
- Inicialización de servicios concurrentes: dentro del subprocesso del servidor, se activan dos componentes críticos de manera paralela. En primer lugar se levanta una aplicación flask (`app.run`) que servirá para la comunicación y monitoreo externo. En segundo, se inicia un hilo dedicado al bot (`botthread`) mediante `thread(start_bot_thread).start()`, con esto separa la ejecución web de la lógica de trading.
- Carga de configuraciones: el `botthread` invoca a la función encargada de leer los parámetros operativos `load_config("config.yaml")`, y de devolver un diccionario de configuración.
- Instanciación del módulo de exchange: con su diseño intercambiable, el bot evalúa la configuración recién cargada. Si existe un archivo mock (`mock_file`), se instancia un cliente de prueba (`mockbinanceclient`) para simulaciones. De lo contrario, asume un entorno de producción o `testnet` y crea un cliente real

de binance (binanceclient) con las claves api proporcionadas.

- Instanciación del módulo de estrategia: el hilo delega la creación del modelo analítico a una fábrica de estrategias (strategyfactory). Dependiendo del nombre en la configuración, la fábrica instancia la clase correspondiente, y carga los parámetros y el modelo subyacente.
- Arranque del motor del bot: finalmente, el botthread inyecta la configuración, el cliente de exchange y la estrategia recién creados dentro de la instancia principal del bot (el motor) y ejecuta su método principal: bot.run().

Fase de ejecución continua

Esta es la fase central operada por el motor del bot, caracterizada por un bucle principal que se repite periódicamente cada tick_interval segundos mientras el flag bot.running sea verdadera. Refleja el ciclo de “recolectar, predecir y actuar” ilustrado en la figura [3.7](#).

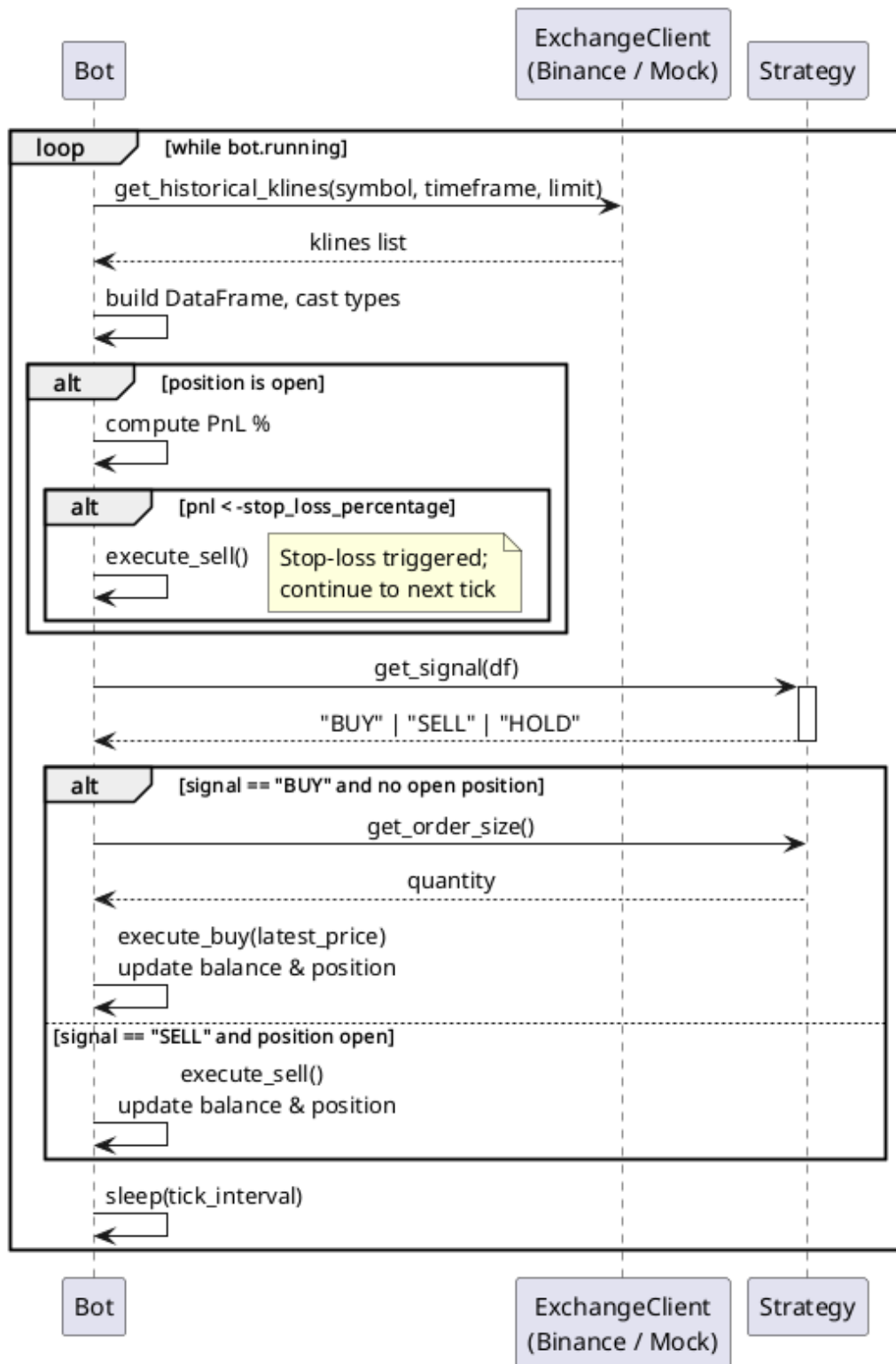


FIGURA 3.7. Diagrama del bucle de principal del bot de trading.

Como vemos en la figura, la ejecución continua consiste en los siguientes pasos:

- Recolección de datos: en cada iteración, el bot solicita al exchange los datos históricos del mercado con una invocación a `get_historical_klines`

con parámetros de símbolo, marco temporal límite (timeframe). El exchange devuelve una lista de velas (klines).

- **Procesamiento de datos:** el bot toma la lista cruda de velas y construye un dataframe, asegurándose de castear correctamente los tipos de datos para que puedan ser consumidos por el modelo matemático.
- **Gestión de riesgos (stop-loss):** antes de buscar nuevas señales, el sistema verifica si existe una posición abierta en el mercado. Si es así, el bot calcula internamente el porcentaje de pérdidas y ganancias (pnl).
- **Control de riesgo:** si el rendimiento es inferior al umbral de riesgo permitido, el bot ejecuta inmediatamente una orden de venta (`execute_sell()`), registra que se activó el stop-loss y finaliza la iteración actual. Después salta directamente al siguiente tick sin consultar a la estrategia.
- **Predicción de señal:** si el stop-loss no se activa, el bot pasa el dataframe procesado al módulo de estrategia mediante la función `get_signal(df)`. Tras evaluar los datos, la estrategia devuelve un veredicto direccional estricto: “buy” (comprar), “sell” (vender) o “hold” (mantener).
- **Ejecución de órdenes de mercado:** el bot evalúa la señal de la estrategia cruzada con su estado actual:
- **Flujo de compra:** si la señal es “buy” y no hay ninguna posición abierta, el bot le consulta a la estrategia el tamaño de la orden (`get_order_size()`). Tras recibir la cantidad, se ejecuta la orden de compra (`execute_buy(latest_price)`) a través del módulo de exchange y actualiza sus balances y estado de posición internamente.
- **Flujo de venta:** si la señal es “sell” y el bot ya posee una posición abierta, procede a ejecutar la venta (`execute_sell()`) e igualmente actualiza el balance de la cuenta y su estado de posición.
- **Pausa táctica:** al finalizar la lógica operativa, el bot suspende su ejecución mediante `sleep(tick_interval)` para evitar saturar las apis del exchange y esperar a la formación de nuevos datos de mercado antes de reiniciar el bucle.

Fase de monitoreo remoto

El diseño, como se puede ver en la figura 3.8, permite auditar el estado del bot en tiempo real sin interrumpir el bucle de trading, gracias a la aplicación web paralela.

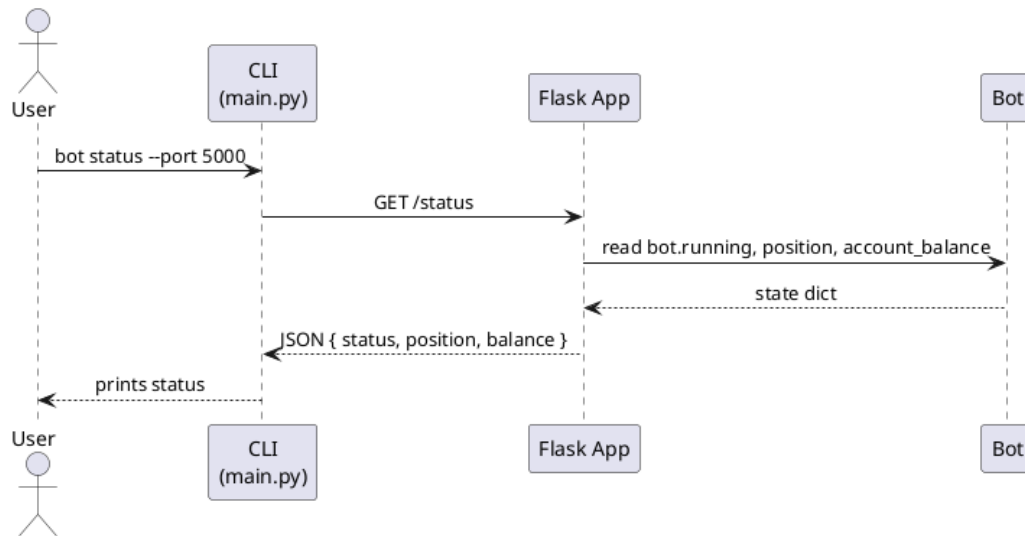


FIGURA 3.8. Diagrama del proceso de monitoreo del bot de trading.

A continuación se describen los pasos del proceso de monitoreo:

- Petición de estado: el usuario ejecuta `bot status -port 5000` en la cli.
- Consulta http: la cli transforma este comando en una petición get dirigida a la ruta `/status` expuesta por la aplicación flask.
- Lectura interna: el servidor flask interactúa con el espacio de memoria del bot, leyendo variables críticas de estado en tiempo real como `bot.running` (si está activo), la posición actual en el mercado y el balance de la cuenta.
- Respuesta al usuario: el bot transfiere este diccionario de estado a flask, que a su vez lo serializa en formato json (conteniendo estatus, posición y balance) y lo envía de regreso a la cli. Finalmente, la cli imprime esta información de forma legible para el usuario.

Fase de apagado

El sistema contempla mecanismos robustos para finalizar el proceso del bot de manera segura. El diagrama 3.9 expone dos vías para detener la operación:

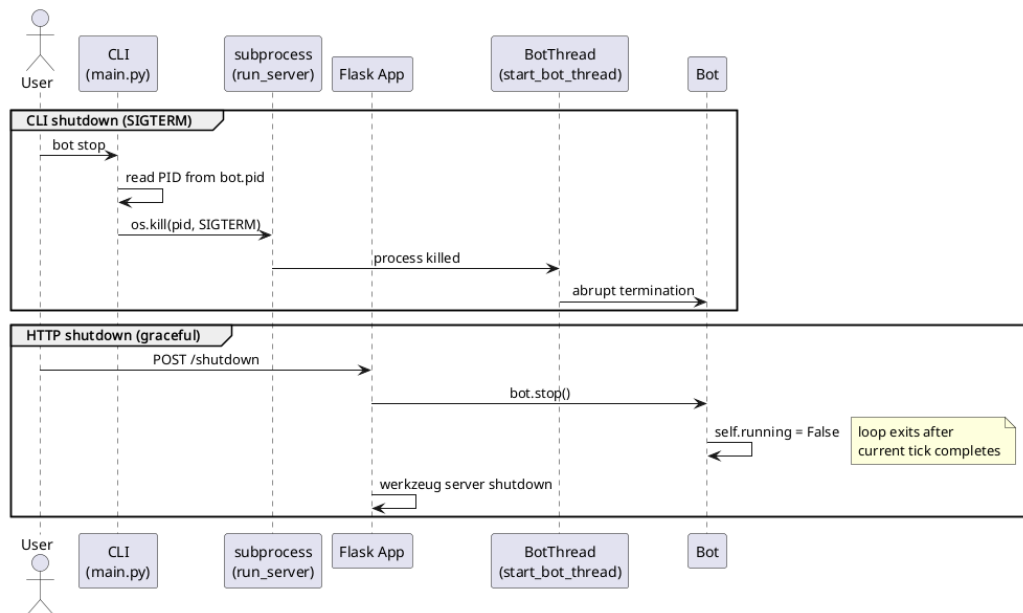


FIGURA 3.9. Diagrama del proceso de apagado del bot de trading.

A continuación se describen los pasos del proceso de apagado:

- Apagado forzado por cli: el usuario emite el comando bot stop. La cli lee el archivo que contiene el identificador del proceso (bot.pid) e interactúa directamente con el sistema operativo mediante el envío de una señal de terminación (`os.kill(pid, sigterm)`) al servidor para que aborte la ejecución.
- Apagado elegante vía api: como método alternativo anotado en el diagrama, se puede enviar una petición post a la ruta /shutdown a través de flask. En este escenario, flask se comunica con el bot con una llamada al método `bot.stop()`, el cual cambia internamente la variable `self.running` a false. Esto permite que el bucle principal finalice su iteración actual de manera limpia, con la lubricación de los hilos y el cierre del servidor de forma orquestada y sin corrupción de estado.

3.8. Diseño del sistema

El sistema esta estructurado siguiendo la temática de pipeline de investigación propuesta por López de Prado en *Advances in financial machine learning*:

1. `data_analysis`: este directorio implementa el paso de curación de datos y de análisis de característica, aquí se proveen rutinas para recopilar, limpiar, indexar, almacenar, y ajustar los datos para entregarlos en un formato útil para el resto de la cadena de producción y también para transformar datos brutos en señales informativas con valor predictivo. En particular se implementan, en el paquete `bar_aggregation`, funciones para agregar barras de tiempo en barras de volumen. Estas incluyen estrategias por volumen de USDT o de BTC. También se encuentran otras utilidades para el preprocesamiento de datos, en el subpaquete `data_analysis`, como ser la diferenciación fraccionaria, operadores de medias móviles. Aquí se encuentran los mas de 25 indicadores de trading usados en este trabajo, como ser

momentum, relative strenght index, MACD, entre otros. Las funciones para el etiquetado, descritas anteriormente, como son, por horizonte fijo, por horizonte ajustado por volatilidad, y de barrera triple, se hacen disponible en el subpaquete `labeling`.

2. `modeling`: en este paquete viven los algoritmos de inversión que utilizan los datos provistos por las subrutinas encontradas en el paquete de `data_analysis` así también como las subrutinas para procesar dichos datos y transformarlos en variables independientes para los modelos. Para los propósitos de este trabajo aquí es donde se encuentran las cuatro categorías de estrategias que se desarrollaron: modelos basados en indicadores de trading, modelos basados en modelos estadísticos tradicionales, modelos basados en predictores clásicos de machine learning, y modelos basados en deep learning, en particular, utiliza la arquitectura de transformers. Para su evaluación aquí también se implementa el algoritmo de k-fold purged cross validation ya descrito anteriormente. En adición a esto también se encuentran utilidades para la reutilización de código dedicado a la interacción con la biblioteca mlflow dedicado a guardar los resultados obtenidos en los experimentos realizados. Con el objetivo de coordinar las interacciones previamente mencionadas y de estandarizar como se alimentan los algoritmos con datos, como se procesan y se orquestaran para su evaluación. Centralizar y reutilizar esta lógica asegura que siempre se van a tomar las mismas medidas y con el mismo criterio para todos los algoritmos estudiados. Va a evitar que, al reescribir código similar, se introduzcan variantes o errores conceptuales a la hora de separar los datos de entrenamiento y de prueba. Como contrapartida, esto obliga a, sin incurrir en una complejidad excesiva, destilar una interfaz lo suficientemente flexible como para ser utilizada por diversas categorías de modelos con distintos requerimientos en lo que respecta a los dominios de estructurado de datos, etiquetado, e ingeniería de características.
3. `backtesting`: aquí se evalúa la rentabilidad de la estrategia bajo diversos escenarios simulados y estiman la probabilidad de sobreajuste del backtest teniendo en cuenta el número de ensayos y errores realizados. Los principales subpaquetes a nombrar son `cpcv` y `cpcv_runner`. En el primero se implementan las funciones puras que calculan los splits combinatorios, la lógica de embargo, y la reconstrucción de caminos para el algoritmo de combinatorially purged cross validation. En el segundo se implementan funciones que unifican la lógica par aplicar CPCV a los modelos existentes.
4. `app`: en este paquete vive el bot de trading tal cual descrito en la sección 3.7.

3.9. Pipeline de modelos

La abstracción central utilizada en el desarrollo de las pruebas para la optimización fue el concepto de pipeline. Como ya se dijo, este fue con el objetivo de centralizar la coordinación de las áreas principales que componen la investigación de estrategias de inversión según López de Prado. En la figura 3.10 se ve conceptualmente el flujo de datos pasar a través del pipeline. Describe dos fases. La fase inicial transforma datos crudos en un formato tabular. El proceso comienza con el estructurado de datos, que genera barras de información. Estas barras

alimentan dos procesos paralelos: la ingeniería de características, que crea variables predictivas, y el etiquetado y ponderación, que define las variables objetivo. Luego, ambas ramas convergen en el paso de alineamiento: lo que garantiza que todas las matrices compartan los mismos índices temporales para formar el dataset final.

La segunda fase utiliza este dataset para modelar y validar la estrategia. Los datos ingresan al módulo para dividir folds: aplica una validación cruzada purgada para evitar filtraciones de información. Este proceso inicia un ciclo repetitivo de n folds. Dentro de cada iteración secuencial ocurren cuatro pasos: el escalado ajusta los datos, la reducción de dimensiones extrae la información relevante, el entrenamiento y predicción ajusta el algoritmo para generar pronósticos, y finalmente la evaluación mide el rendimiento frente a datos reales. Así se conecta el procesamiento analítico con una validación rigurosa del modelo.

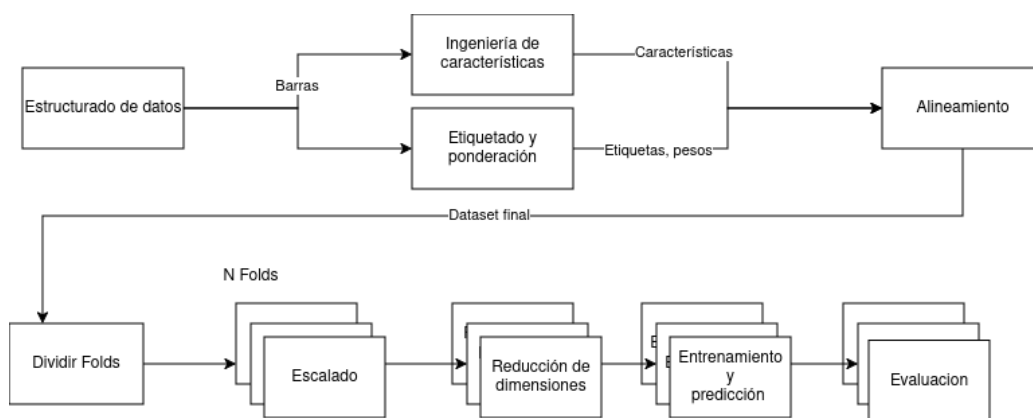


FIGURA 3.10. Flujo de datos en distintas etapas de procesamiento del pipeline.

Con esta estructura de base se pudieron evaluar los distintos modelos y estrategias de forma consistente mediante el uso de herencia como se ve en la figura 3.11.

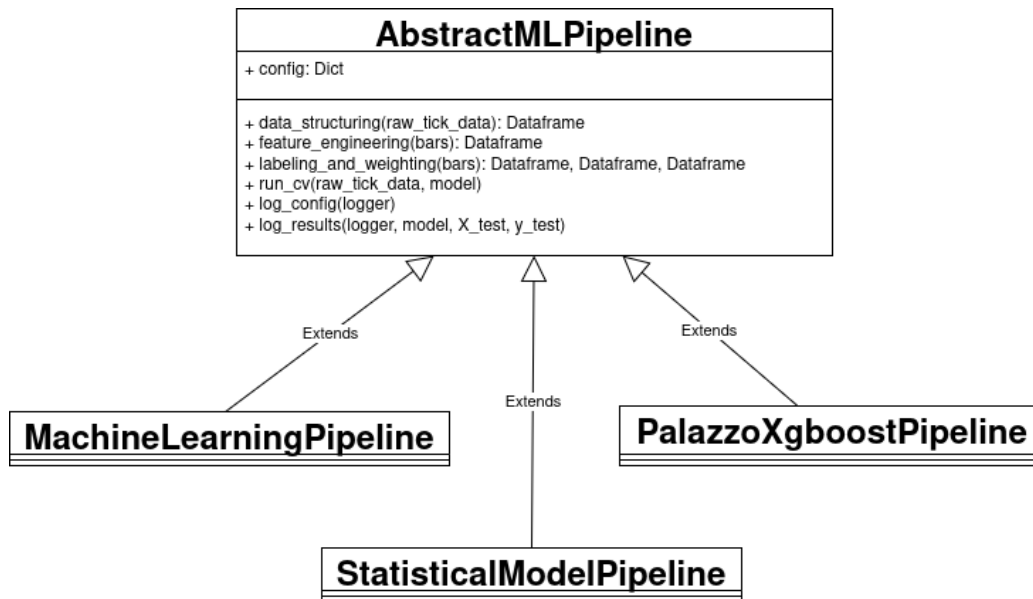


FIGURA 3.11. Estructura jerárquica de herencia de clase para AbstractMLPipeline.

3.10. Estructura de pruebas realizadas con Chronos

Se aprovecha la flexibilidad brindada por la estructura de clases descrita en la sección 3.9 (véase figura 3.12) para realizar una serie de pruebas centradas en la familia de modelos Chronos. Las pruebas se pueden dividir conceptualmente en dos grandes categorías: las que usan Chronos para realizar la predicción y las que usan chronos para hacer transfer learning.

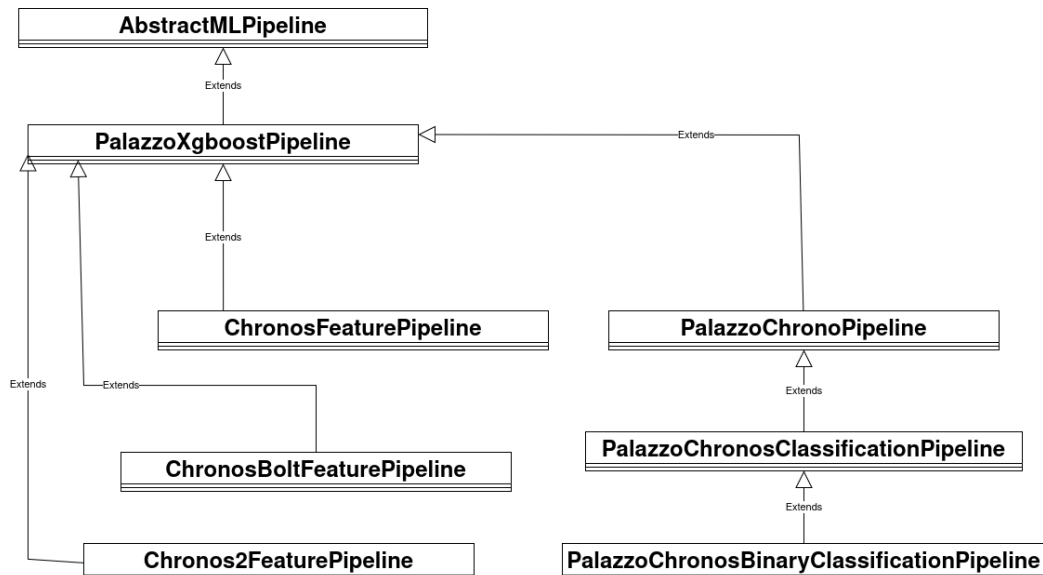


FIGURA 3.12. Estructura jerárquica de herencia de clase para los experimentos realizados con Chronos.

En la primera categoría se utiliza el modelo chronos como regresor o como clasificador. En forma de regresor se tiene clases PalazzoChronosPipeline que

trata de predecir el precio directamente. Del lado de los clasificadores se tienen `PalazzoChronosClassificationPipeline` que realiza una regresión sobre el precio de cierre pero luego solo se toma si es mayor o menor al cierre anterior, en efecto transformándolo en un clasificador; y `PalazzoChronosBinaryClassificationPipeline` al que, en vez de alimentarlo con los precios de cierre, se le proveen directamente las etiquetas +1 y -1 que indican si el precio será mayor o menor que el anterior.

En la categoría de transfer learning se tiene a la familia de los `ChronosFeaturePipelines` que modifican el pipeline de inspirado en el trabajo de Palazzo, que se usó como línea de base, para agregar los embeddings de chronos como características para que sean usadas por un modelo tabular.

En ambas categorías se realizaron pruebas con inferencia de zero-shot y también luego de hacer fine-tuning. Para éste último se utilizó una estrategia de low rank adaptation lo que hizo factible la tarea en el hardware disponible.

Capítulo 4

Ensayos y resultados

En este capítulo se exponen los datos con los que se trabajó y se expande sobre los resultados obtenidos en base a los experimentos comentados en el capítulo anterior. Se los exponen en el formato en que se investigaron, de más simple a más complejo. Se finaliza con el objetivo principal de este trabajo que son los modelos de transformers.

4.1. Datos utilizados

Como ya se mencionó antes, en este trabajo se utilizaron precios de cierre de Bitcoin en relación a USDT entre 2020-01-01 y 2025-11-07. En la tabla 4.1 se ve el rendimiento de bitcoin durante el periodo analizado.

TABLA 4.1. Resumen de rendimiento de la estrategia Buy and Hold entre 2020-01-01 y 2025-11-07.

Métrica	Valor
Retorno Total	31995,25 %
Volatilidad anualizada	70,72 %
Ratio de Sharpe	1,16
Drawdown máximo	-83,27 %

También se ilustra como las barras de volumen afectan a la serie de tiempo original (figura 4.1). Se aprecia como la forma general sigue presente pero, en cierta manera, un poco más suavizada. Se pueden ver que los movimientos pequeños se condensan en un solo punto.

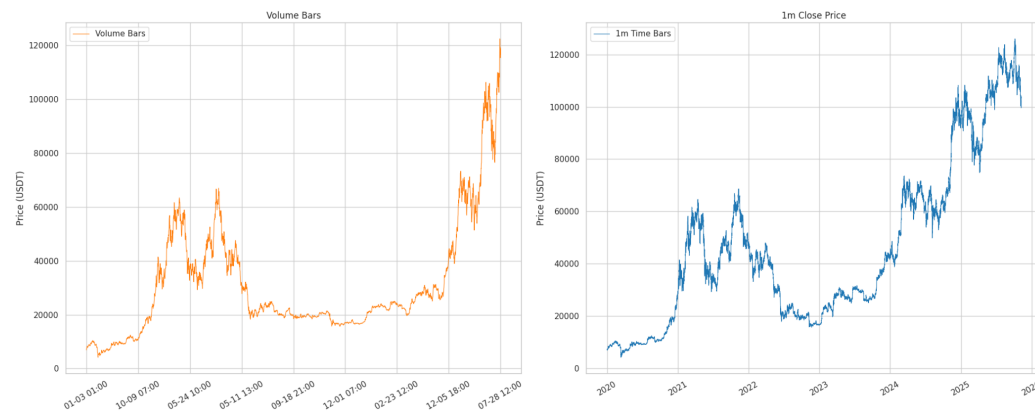


FIGURA 4.1. Comparativa de barras de volumen y precio de cierre con frecuencia de 1m.

La figura 4.2 ilustra, además, cómo el agregado por barras de volumen cambia la velocidad de muestreo con respecto un incremento en la varianza. Esto se ejemplifica bien si se compara el periodo que va del 9 de enero y al 13 de enero, con el que va del 13 al 21 del mismo mes. Tienen la misma cantidad de puntos (seis) pero el segundo período cubre el doble de tiempo.



FIGURA 4.2. Superposición de precio de cierre cada 1m, barras de volumen a 50 000 unidades de BTC, y volumen de BTC.

4.2. Pruebas con algoritmos de trading

En esta sección se hizo un análisis sobre las estrategias basadas en algoritmos de trading. Esto es, se codificaron reglas fijas basadas en distintos indicadores y, en base a ellas, se determinó una señal de compra o venta para la ocasión actual.

Configuración: CV purgado de 5 pliegues con 1 % de embargo, 0,1 % de comisión, optimización Optuna en datos de 1 minuto de BTCUSD.

Como se observa en la tabla 4.2, el `BollingerBandsClassifier` tiene el techo de rendimiento más alto. Por otro lado, la tabla 4.3 muestra que el

RSIDivergenceClassifier es la estrategia más confiable: presenta el Sharpe medio más alto y una varianza extremadamente baja.

TABLA 4.2. Resultados de la mejor optimización basados en el promedio pesado de F1 en validación cruzada (*avg_cv_weighted_f1*).

Rank	Clasificador	Mejor F1	Mejores Parámetros
1	BollingerBandsClassifier	0,5108	bb_window=21, bb_std=1,77
2	RSIDivergenceClassifier	0,5079	rsi_window=9, divergence_period=23
3	MaCrossoverClassifier	0,5012	short=6, long=7
4	MultiIndicatorClassifier	0,4986	fast_sma=42, slow_sma=227, ...
5	MACDClassifier	0,4985	fast=5, slow=6, signal=17
6	SmaCrossClassifier	0,4981	n1=47, n2=199

TABLA 4.3. Distribución del ratio de Sharpe a nivel de prueba (184 pruebas totales).

Clasificador	N	Media	Std	Min	Median	Max
RSIDivergenceClassifier	32	0,5072	0,0008	0,5051	0,5077	0,5079
BollingerBandsClassifier	30	0,5043	0,0221	0,3882	0,5093	0,5108
SmaCrossClassifier	30	0,4963	0,0019	0,4909	0,4968	0,4981
MACDClassifier	41	0,4957	0,0019	0,4924	0,4957	0,4985
MaCrossoverClassifier	30	0,4956	0,0024	0,4919	0,4950	0,5012
MultiIndicatorClassifier	21	0,4899	0,0270	0,3731	0,4973	0,4986

La tabla 4.4 cuantifica esta consistencia y muestra el porcentaje de pruebas que supera distintos umbrales de Sharpe.

TABLA 4.4. Consistencia: porcentaje de pruebas que superan los umbrales de Sharpe establecidos.

Clasificador	>0,49	>0,495	>0,500	>0,505	>0,510
RSIDivergenceClassifier	100 %	100 %	100 %	100 %	0 %
BollingerBandsClassifier	97 %	97 %	97 %	77 %	37 %
SmaCrossClassifier	100 %	77 %	0 %	0 %	0 %
MACDClassifier	100 %	71 %	0 %	0 %	0 %
MaCrossoverClassifier	100 %	50 %	7 %	0 %	0 %
MultiIndicatorClassifier	81 %	71 %	0 %	0 %	0 %

La tabla 4.5 resume la sensibilidad de los parámetros clave para cada clasificador.

Conclusión: RSIDivergence y BollingerBands son los dos mejores exponentes. El primero es la opción segura para implementación por su baja varianza (ver tabla 4.4), mientras que las bandas de Bollinger ofrecen mayor recompensa a cambio de un riesgo superior.

TABLA 4.5. Sensibilidad de parámetros (correlación con *avg_cv_sharpe*).

Clasificador	Parámetro	Corr	Interpretación
SmaCrossClassifier	n1	+0,89	Ventana lenta más larga → mejor
MaCrossoverClassifier	long_window	-0,58	Ventanas cortas funcionan mejor
BollingerBandsClassifier	bb_std	-0,40	Se prefieren bandas estrechas
MACDClassifier	slow_span	-0,92	Spans cortos fuertemente preferidos
RSIDivergenceClassifier	rsi_window	-0,76	Ventana RSI corta es mejor (best: 9)

4.3. Pruebas con algoritmos estadísticos clásicos

En esta sección se hizo un análisis sobre las estrategias basadas en algoritmos tradicionales de series de tiempo.

Configuración: CV purgado de 3 pliegues, 1 % de embargo, datos de 1h de BT-CUSDT desde 2020-01-01. Se evaluaron 5 familias de modelos.

Como se detalla en la tabla 4.6, el `ARIMAClassifier` es el mejor modelo estadístico evaluado: alcanza un F1 máximo de 0,5004. Sin embargo, su alta varianza ($\text{std} = 0,118$) y sus modos de falla catastróficos lo hacen poco confiable en la práctica, como se evidencia en la tabla 4.8.

TABLA 4.6. Mejor resultado de optimización por familia de modelo basado en el promedio de F1 en validación cruzada (*avg_cv_f1*).

Rank	Clasificador	Mejor F1	n_trials
1	ARIMAClassifier	0,5004	30
2	ARIMAXGARCHClassifier	0,4954	10
3	SARIMAClassifier	~0,4862	4
4	KalmanARIMAClassifier	0,4184	10

Los modelos de series temporales muestran una inestabilidad estructural. La tabla 4.7 revela que el parámetro `threshold` es el factor determinante: las familias ARIMA solo funcionan cuando se configuran para predecir retornos esperados extremadamente pequeños ($\text{umbral} \sim 1e - 6$).

TABLA 4.7. Análisis de sensibilidad: Correlación entre hiperparámetros clave y el rendimiento (F1).

Modelo	Parámetro	Corr	Interpretación
ARIMA-family	threshold	-0,93	El umbral cercano a cero es crítico.
ARIMAXGARCH	threshold	-0,99	Umbrales altos degradan el modelo.
ARIMAClassifier	lookback_len	-0,37	Se prefiere historial corto (720h).
ARIMAXGARCH	q (ARIMA MA)	+0,85	Orden MA alto es significativamente mejor.

Además de esta limitación, los modelos ARIMA presentan modos de falla catastróficos. Las altas desviaciones estándar reportadas en la tabla 4.8 (0,118 para

ARIMA, 0,116 para GARCH y 0,165 para Kalman) son un claro indicador de inestabilidad. Las combinaciones deficientes de parámetros producen puntuaciones F1 cercanas a cero, lo que los hace poco confiables para aplicaciones prácticas. Como evidencia de esto, ARIMA solo logra un $F1 \geq 0,48$ en el 47 % de las pruebas, mientras que Kalman nunca logra alcanzar esa marca.

TABLA 4.8. Distribución de la métrica F1 a través de todas las pruebas realizadas por clasificador.

Clasificador	Media	Std	Min	Median	Max	$\geq 0,50$
SARIMAClassifier	0,4605	0,0457	0,3922	0,4819	0,4862	0 %
ARIMAClassifier	0,4382	0,1183	0,0214	0,4768	0,5004	3 %
ARIMAXGARCHClassifier	0,3855	0,1159	0,1880	0,4062	0,4954	0 %
KalmanARIMAClassifier	0,2476	0,1653	0,0094	0,3117	0,4184	0 %

Queda en evidencia que, en este contexto, añadir complejidad perjudica el rendimiento. Al contrastar los resultados obtenidos, un modelo ARIMA (6 parámetros) supera a los más complejos ARIMAX-GARCH (8 parámetros) y Kalman-ARIMA (6 parámetros). Cada nueva capa de complejidad reduce el F1 promedio y aumenta la varianza de los resultados. El filtro de Kalman, en particular, parece degradar la señal original del modelo ARIMA en lugar de potenciarla.

En cuanto a la parametrización, la mejor configuración de ARIMA converge de manera muy ajustada. Las cinco mejores pruebas de este modelo comparten exactamente los mismos valores de base ($p = 4$, $d = 0$, $q = 1$, $\text{refit} = 720\text{h}$, $\text{lookback} = 720\text{h}$), y difieren únicamente en el umbral. Esto sugiere que el orden autorregresivo y de media móvil de ARIMA está bien identificado para este activo, pero que el espacio de búsqueda inicial para el umbral fue demasiado amplio.

Conclusión: podemos afirmar con seguridad que se logró un resultado subóptimo aunque esperable. Los algoritmos estadísticos no supieron diferenciar la escasa señal del preponderante ruido presente en los mercados financieros. En resultados concretos el mejor f1 no pudo superar el de un intento aleatorio y en sharpe ratio quedó muy por debajo de la base del buy and hold.

En el detalle de los modelos estudiados, todos los algoritmos estadísticos de series temporales (ARIMA, GARCH y Kalman) resultan estructuralmente inestables para este conjunto de datos. Exigen una calibración del umbral extremadamente precisa e, incluso en el mejor de los escenarios (ARIMAClassifier con $p = 4$, $d = 0$, $q = 1$), apenas logran superar el rendimiento de un clasificador aleatorio.

4.4. Pruebas con algoritmos de machine learning

En esta sección se realizan las primeras tareas significativas de ingeniería de características y de etiquetado, ya que las estrategias anteriores recibían solamente el precio de cierre. También en esta etapa se definió la estrategia de *baseline*, inspirada en el trabajo de Palazzo. Los experimentos con modelos transformer (Chronos y variantes) se presentan en la sección siguiente.

Configuración: CV purgado de 3 pliegues con 1 % de embargo, datos de 1 minuto de BTCUSDT en el rango previamente dicho. La métrica primaria es el F1 macro promedio entre pliegues (*avg_cv_f1*). El etiquetado sigue el esquema de barras de volumen con triple barrera: $\text{bar_return}[t+1] \geq \text{bar_return}[t] + \text{intra_bar_std}[t] \times \tau$.

La tabla 4.9 describe los tres experimentos evaluados.

TABLA 4.9. Pipelines y modelos evaluados en la etapa de machine learning.

Exp.	Pipeline	Modelo	Datos
59	PalazzoXGBoostPipeline	XGBoost (CUDA)	1-min consolidado
61	PalazzoAutoGluonPipeline	AutoGluon	1-min consolidado
62	PalazzoXGBoostPipeline (PR)	XGBoost (CUDA)	1-min consolidado

La tabla 4.10 muestra la distribución del F1 para cada algoritmo a través del conjunto completo de pruebas.

TABLA 4.10. Distribución del F1 macro a través de todas las pruebas por algoritmo.

Pipeline	n	Min	p25	Mediana	p75	Max	Std
PalazzoXGBoostPipeline (PR)	30	0,569	0,609	0,624	0,637	0,650	0,019
PalazzoXGBoostPipeline	60	0,584	0,606	0,612	0,623	0,647	0,015
PalazzoAutoGluonPipeline	20	0,561	0,606	0,616	0,618	0,639	0,016

Los tres pipelines se agrupan en una banda de 0,011 puntos de F1 (0,639–0,650). PalazzoXGBoostPipeline (PR) y PalazzoXGBoostPipeline son prácticamente equivalentes: la variante PR añade características de reversión de precio y obtiene +0,003 en el mejor resultado y +0,008 en la mediana, ambas diferencias dentro de una desviación estándar. PalazzoAutoGluonPipeline (mejor F1=0,639) se aproxima a PalazzoXGBoostPipeline (mejor F1=0,647) con una diferencia de solo 0,008, pero con un costo de cómputo aproximadamente 10 veces mayor; el mejor modelo seleccionado por AutoGluon fue `LightGBMXT_BAG_L1`.

Los patrones transversales observados en todos los experimentos son:

- **use_pca=False:** universal en los tres pipelines; la compresión PCA elimina señal discriminativa.
- **max_depth XGBoost:** diverge — 4 en PalazzoXGBoostPipeline y 19 en la variante PR; el conjunto PR admite divisiones más profundas por sus características adicionales.
- **n_estimators y tasa de aprendizaje XGBoost:** 373 estimadores en PalazzoXGBoostPipeline frente a 278 en PR; ambas variantes convergen a una tasa de aprendizaje de 0,003–0,004, coherente con un dataset de 3M+ filas que requiere alta regularización.
- **τ y volume_threshold:** PR ($\tau=1,289$, vol.=62.855), Palazzo ($\tau=1,261$, vol.=28.522), AutoGluon ($\tau=0,980$, vol.=32.034). Los tres convergen a barrera amplia ($\tau > 0,98$); volume_threshold varía de 28.522 a 62.855, lo que indica sensibilidad a la granularidad de las barras de volumen.

Conclusión: Los tres pipelines convergen en la banda 0,639–0,650 de F1 macro. `PalazzoXGBoostPipeline` (PR, F1=0,650) y `PalazzoXGBoostPipeline` (F1=0,647) constituyen los mejores resultados de esta etapa con una separación estadísticamente no significativa entre sí. `PalazzoAutoGluonPipeline` (F1=0,639) es competitivo pero requiere un costo computacional aproximadamente 10 veces mayor. Cabe señalar que `PalazzoXGBoostPipeline` fue actualizado después de estos experimentos para incluir SMOTE y selección de características por correlación de Pearson por pliegue, por lo que estos resultados preceden a esa mejora.

4.5. Pruebas con Chronos

En esta sección se presenta el análisis detallado de los experimentos con Chronos, organizados en tres familias de arquitectura con objetivos distintos: *embedding con clasificador*, *pronóstico directo* y *meta-etiquetado*. La tabla 4.11 describe las siete clases de pipeline implementadas.

TABLA 4.11. Siete pipelines Chronos distribuidos en tres familias de arquitectura.

Familia	Pipeline	Arquitectura
Embedding + Clasificador	<code>ChronosFeaturePipeline</code>	Encoder T5 → pool → tabulares → AutoGluon
Embedding + Clasificador	<code>Chronos2FeaturePipeline</code>	Parches Chronos-2 → pool → tabulares → AutoGluon
Embedding + Clasificador	<code>ChronosBoltFeaturePipeline</code>	Parches BOLT → pool → tabulares → AutoGluon
Embedding + Clasificador	<code>FinetunedChronosFeaturePipeline</code>	Ajuste fino T5 por pliegue → embeddings → AutoGluon
Pronóstico directo	<code>PalazzoChronosClassificationPipeline</code>	Pronóstico de precio rodante → señal de dirección
Pronóstico directo	<code>PalazzoChronosBinaryClassificationPipeline</code>	Ajuste fino en etiquetas {+1, -1} → umbral en 0
Meta-etiquetado	<code>ChronosMetaLabelingPipeline</code>	Modelo primario → meta-modelo sobre aciertos

Configuración general: CV purgado de 3 pliegues con 1 % de embargo para las familias de embedding y meta-etiquetado, y 5 % de embargo para pronóstico directo. Datos: 1 minuto de BTCUSD entre 2020-01-01 y 2025-10-08 (3.033.173 filas) para todas las variantes. Métrica primaria: F1 macro promedio entre pliegues (*avg_cv_f1*) para embedding y meta-etiquetado; *avg_cv_score* para pronóstico directo. Estas métricas no son directamente comparables entre familias.

Familia 1: Embedding con clasificador. Cada pipeline genera representaciones vectoriales densas de una ventana de precios de cierre recientes mediante un encoder Chronos, y concatena esos embeddings con las características tabulares de `PalazzoXGBoostPipeline`. AutoGluon clasifica la matriz combinada. La tabla 4.12 resume los cuatro experimentos de esta familia y la tasa de éxito de cada uno.

TABLA 4.12. Experimentos con embedding + clasificador y análisis de pruebas fallidas.

Exp.	Pipeline	Válidas	Total	Causa de fallo
73	<code>ChronosFeaturePipeline</code>	10	10	—
71	<code>ChronosBoltFeaturePipeline</code>	10	10	—
67	<code>FinetunedChronosFeaturePipeline</code>	4	10	Checkpoint bolt-tiny incompatible con ajuste fino
65	<code>Chronos2FeaturePipeline</code>	3	10	Variantes tiny y mini: fallo por memoria o crash

La tabla 4.13 presenta la distribución del F1 macro. Se incluyen los resultados de estudios Optuna anteriores y de corridas únicas previas al experimento formal, ya que superan los del experimento documentado con límite de 600 segundos.

TABLA 4.13. Distribución del F1 macro en los experimentos de embedding. ^a Estudio Optuna con `time_limit` libre. ^b Corridas únicas sin optimización formal (exp. 21).

Pipeline	n	Min	p25	Mediana	p75	Max	Std
ChronosFeaturePipeline (exp. 73)	10	0,573	0,579	0,605	0,624	0,633	0,022
ChronosFeaturePipeline (estudio ant.) ^a	20	—	—	—	—	0,653	—
ChronosFeaturePipeline (corrida única) ^b	5	0,632	—	—	—	0,650	—
ChronosBoltFeaturePipeline (exp. 71)	10	0,598	0,607	0,609	0,613	0,628	0,008
ChronosBoltFeaturePipeline (estudio ant.) ^a	10	—	—	—	—	0,644	—
Chronos2FeaturePipeline (exp. 65)	3	0,604	0,610	0,615	0,618	0,621	0,007
FinetunedChronosFeaturePipeline (exp. 67)	4	0,598	0,599	0,604	0,610	0,611	0,005

Los cuatro pipelines se agrupan en la banda 0,611–0,653. ChronosFeaturePipeline (T5) obtiene el mayor techo y la mayor dispersión (std=0,022), lo que indica que el espacio de búsqueda aún tiene recorrido. El experimento formal (exp. 73) usa un paso de 64 en la dimensión de ventana a partir de 32, por lo que la ventana=128 nunca se evalúa en la optimización; una corrida única con `t5-tiny` y ventana=128 alcanza F1=0,650, que iguala al mejor resultado global. Los estudios Optuna anteriores, con `time_limit` como hiperparámetro libre, encuentran que `time_limit=300` supera a `time_limit=600` para el mismo preset (*high_quality*), posiblemente porque AutoGluon concentra los recursos en su learner más fuerte en lugar de construir ensambles más profundos que sobreajustan.

ChronosBoltFeaturePipeline muestra la menor dispersión de todos los pipelines (std=0,008), lo que refleja la estabilidad de los embeddings BOLT. Su techo en 0,628 (exp. 71) y 0,644 (estudio anterior) se sitúa por debajo de T5. Chronos2FeaturePipeline solo funciona de forma fiable con la variante `chronos-2-small`; las variantes `tiny` y `mini` fallan en 7 de 10 pruebas por falta de memoria o crash, lo que deja únicamente 3 puntos de datos válidos y un techo aparente de 0,621. FinetunedChronosFeaturePipeline obtiene el peor resultado de la familia (0,611) pese al costo computacional de ajustar un modelo Chronos por pliegue de CV: la señal binaria sobre barras de volumen resulta demasiado ruidosa para mejorar las representaciones preentrenadas de T5-tiny en el tiempo disponible. El pipeline además solo acepta checkpoints de familia T5; los modelos BOLT finalizan con estado FINISHED pero no registran métricas.

El backtest CPCV de ChronosFeaturePipeline (exp. 32) obtiene *f1_mean* entre 0,516 y 0,625, valor próximo al mejor resultado de validación cruzada formal (0,633). Esta concordancia indica que las estimaciones de CV de los pipelines de embedding son realistas y no sobreajustan al esquema de validación.

Familia 2: Pronóstico directo. Estos pipelines usan Chronos de forma autorregresiva para predecir el siguiente precio o una etiqueta {+1, -1}, y convierten la salida continua en una señal binaria. La tabla 4.14 resume los tres experimentos de esta familia.

TABLA 4.14. Experimentos con pronóstico directo Chronos y sus resultados.

Exp.	Pipeline	Enfoque	Resultado
17	PalazzoChronosClassificationPipeline	Precio→dirección (cero-shot)	<i>avg_cv_f1</i> : 0,492–0,494
18	PalazzoChronosBinaryClassificationPipeline	Ajuste fino en {+1, -1}	<i>test_macro_f1</i> : 0,373–0,820
69	PalazzoChronosBinaryClassificationPipeline	Optuna con <code>chronos-2-small</code>	<i>avg_cv_score</i> : 0,803– 0,818

El enfoque de cero-shot (exp. 17) convierte el pronóstico de nivel de precio en una señal de dirección; como el modelo minimiza el error de nivel y no la dirección, el F1 apenas supera 0,50. El ajuste fino binario sin optimización (exp. 18) presenta una variabilidad extrema entre corridas (0,373–0,820) que indica inestabilidad del proceso de ajuste.

El experimento 69, con `chronos-2-small` y 5 pruebas Optuna, registra *avg_cv_score* entre 0,803 y 0,818 (*test_macro_f1* entre 0,804 y 0,819). La mejor prueba converge a $\tau = 0,91$ y `volume_threshold=45.988`. Este resultado supera en más de 0,17 puntos el techo de los pipelines de embedding, pero no es comparable con ellos: la métrica es *avg_cv_score* (distinta de *avg_cv_f1*), el embargo es 0,05 frente a 0,01 en embedding, y la longitud de predicción es 2. Ninguna variante de esta familia cuenta con validación CPCV, requisito mínimo antes de extraer conclusiones sobre su superioridad.

Familia 3: Meta-etiquetado. Un modelo primario predice la señal de dirección; un meta-modelo predice si el primario acierta, y solo se ejecutan las operaciones donde ambos coinciden. La tabla 4.15 resume los resultados de los tres experimentos de esta familia, incluyendo el backtest CPCV del experimento 29.

TABLA 4.15. Resultados de los experimentos con la familia de meta-etiquetado.

Exp.	Pipeline	F1 base	F1 meta	Observación
19	PalazzoMetaLabelingPipeline	0,608	0,587–0,619	Sin componente Chronos (línea de base tabular)
22	ChronosMetaLabelingPipeline	0,655	0,571–0,657	T5-tiny, 8 corridas
29	CPCV de exp. 22	—	0,507–0,609	Backtest con 3 configuraciones

El modelo primario de `ChronosMetaLabelingPipeline` (exp. 22) obtiene un F1 ponderado de 0,655 en todas las corridas, superior al pipeline puramente tabular (0,608), lo que confirma el aporte de los embeddings Chronos en la primera capa. Sin embargo, el meta-filtro no añade valor sistemático: en 6 de 8 corridas degrada el F1, y el rango 0,571–0,657 no supera de forma consistente al modelo primario. El backtest CPCV (exp. 29) obtiene *f1_mean* entre 0,507 y 0,609 en tres configuraciones, coherente con la tendencia observada en validación cruzada. La puntuación interna del meta-modelo de AutoGluon (0,789) se infla porque se evalúa sobre datos fuera de muestra en distribución, no sobre datos de mercado reales.

La tabla 4.16 resume los patrones transversales observados en las tres familias de experimentos.

TABLA 4.16. Patrones y constantes observadas en todos los experimentos con Chronos.

Patrón	Observación
<code>use_pca=False</code>	Universal en todos los experimentos. La compresión PCA destruye señal discriminativa, consistente con el resto de los algoritmos evaluados.
τ fijo	Se fija en 0,70 en todos los experimentos de embedding y meta-etiquetado. El optimizador de pronóstico directo converge a $\tau \approx 0,91$ para <code>chronos-2-small</code> .
<code>volume_threshold</code>	Fijo en 50.000 en embedding. El optimizador de pronóstico directo converge a ≈ 46.000 .
Canal de entrada	Todos los pipelines alimentan solo el precio de cierre a Chronos. Volumen, VWAP y OHLC se ignoran como series de tiempo.
Embeddings como adición	Los embeddings Chronos se concatenan con las características tabulares de <code>PalazzoXGBoostPipeline</code> ; no las reemplazan.
Restricción de ajuste fino	Solo los checkpoints de familia T5 son compatibles con ajuste fino. Los modelos BOLT finalizan con estado FINISHED sin registrar métricas.

Conclusión: La familia de embedding con clasificador es el enfoque más sólido dentro de los experimentos Chronos. `ChronosFeaturePipeline` (T5) obtiene el mayor techo ($F1=0,653$) y la mayor dispersión, con configuraciones clave aún sin explorar, en particular T5-small con ventana=128; su backtest CPCV (exp. 32, $f1_mean$ hasta 0,625) confirma que las estimaciones de CV son realistas. El pronóstico directo con `chronos-2-small` registra avg_cv_score hasta 0,818, el resultado numérico más alto de todas las etapas, pero no cuenta con validación CPCV ni emplea la misma métrica que el resto, por lo que no es directamente comparable. El meta-etiquetado no agrega valor sistemático sobre el modelo primario: la capa adicional impone un balance precisión-exhaustividad que degrada el F1 macro en la mayoría de las corridas. El ajuste fino por pliegue resulta costoso y frágil sin beneficio demostrable. Los parámetros τ y `volume_threshold` permanecen sin explorar en todos los experimentos de embedding y constituyen la dimensión con mayor potencial para trabajos futuros.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] James Chen. *What Is a Trader, and What Do Traders Do?* <https://www.investopedia.com/terms/t/trader.asp>. Visitado el 23-09-2023. 2023.
- [2] Charles D. Kirkpatrick II y Julie R. Dahlquist. *Technical Analysis: The Complete Resource for Financial Market Technicians*. Financial Times Press, 2006. ISBN: 978-0-13-153113-0.
- [3] Qingsong Wen et al. «Transformers in Time Series: A Survey». En: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23)*. 2023, págs. 6778-6786. DOI: [10.24963/ijcai.2023/759](https://doi.org/10.24963/ijcai.2023/759). URL: <https://doi.org/10.24963/ijcai.2023/759>.
- [4] Igor Halperin. *Non-Linear and Meta-Stable Dynamics in Financial Markets: Evidence from High Frequency Crypto Currency Market Makers*. 2025. arXiv: [2509.02941](https://arxiv.org/abs/2509.02941) [q-fin.ST]. URL: <https://arxiv.org/abs/2509.02941>.
- [5] Eugene F. Fama. «Efficient Capital Markets: A Review of Theory and Empirical Work». En: *The Journal of Finance* 25.2 (1970), págs. 383-417.
- [6] Lucas Coelho e Silva, Gustavo de Freitas Fonseca y Paulo Andre L. Castro. «Transformers and attention-based networks in quantitative trading: a comprehensive survey». En: *Proceedings of the 5th ACM International Conference on AI in Finance*. ICAIF '24. New York, NY, USA: ACM, 2024, págs. 822-830. DOI: [10.1145/3677052.3698684](https://doi.org/10.1145/3677052.3698684). URL: <https://doi.org/10.1145/3677052.3698684>.
- [7] *Finbrain Technologies*. <https://finbrain.tech>. Visitado el 11-04-2026.
- [8] Joel Paul. «Financial Time Series Analysis with Transformer Models». En: *ResearchGate* (2024). URL: https://www.researchgate.net/publication/387524930_Financial_Time_Series_Analysis_with_Transformer_Models.
- [9] Marcos López de Prado. *Advances in Financial Machine Learning*. Hoboken, New Jersey: Wiley, 2018.
- [10] Kristina Zucchi. *Derivatives 101*. <https://www.investopedia.com/articles/optioninvestor/10/derivatives-101.asp>. Visitado el 23-09-2023. 2022.
- [11] Ernest P. Chan. *Algorithmic Trading: Winning Strategies and Their Rationale*. John Wiley & Sons, 2013.
- [12] Robert Rhea. *The Dow Theory*. New York: Barron's, 1932.
- [13] Adam Hayes. *Technical Analysis: What It Is and How to Use It in Investing*. <https://www.investopedia.com/terms/t/technicalanalysis.asp>. Visitado el 26-09-2023. 2022.
- [14] José Francisco López. *Teoría de Dow*. <https://economipedia.com/definiciones/teoria-de-dow.html>. Visitado el 26-09-2023. 2018.
- [15] Stefan Jansen. *Machine Learning for Algorithmic Trading - Second Edition*. Packt, 2020. ISBN: 978-1-83921-771-5.
- [16] Gaurav Singhal. *Ensemble Methods in Machine Learning: Bagging Versus Boosting*. <https://www.pluralsight.com/guides/ensemble-methods--bagging-versus-boosting>. Visitado el 31-10-2023. 2020.
- [17] Ashish Vaswani et al. «Attention Is All You Need». En: *CoRR* abs/1706.03762 (2017). arXiv: [1706.03762](http://arxiv.org/abs/1706.03762). URL: <http://arxiv.org/abs/1706.03762>.

- [18] Abdul Fatir Ansari et al. *Chronos: Learning the Language of Time Series*. 2024. arXiv: [2403.07815](https://arxiv.org/abs/2403.07815) [cs.LG].
- [19] William F. Sharpe. «Mutual Fund Performance». En: *The Journal of Business* 39.1 (1966), págs. 119-138.
- [20] Frank A. Sortino y Robert van der Meer. «Downside Risk». En: *Journal of Portfolio Management* 17.4 (1991), págs. 27-31.
- [21] Terry W. Young. «Calmar Ratio: A Smoother Tool». En: *Futures* 20.11 (1991).
- [22] Guilherme Palazzo. «Predicting BTC cryptocurrency price movement in a pre-defined trading volume window using Machine Learning models». Advisor: Prof. PhD. Elton Felipe Sbruzzi. Co-advisor: Prof. PhD. Michel Carlo Rodrigues Leles. Dissertation of Master of Science. São Jos'e dos Campos: Instituto Tecnol'ogico de Aeron'utica y Universidade Federal de São Paulo, 2023, 73f.
- [23] Optuna. <https://optuna.org/>. Visitado el 9-05-2026.